

PROCESSOS DE DESENVOLVIMENTO E SUSTENTAÇÃO DE SOFTWARE

Processo Ágil, Scrum, Kanban, Lean



Livro Eletrônico

Presidente: Gabriel Granjeiro

Vice-Presidente: Rodrigo Calado

Diretor Pedagógico: Erico Teixeira

Diretora de Produção Educacional: Vivian Higashi

Gerente de Produção Digital: Bárbara Guerra

Coordenadora Pedagógica: Élica Lopes

Todo o material desta apostila (incluindo textos e imagens) está protegido por direitos autorais do Gran. Será proibida toda forma de plágio, cópia, reprodução ou qualquer outra forma de uso, não autorizada expressamente, seja ela onerosa ou não, sujeitando-se o transgressor às penalidades previstas civil e criminalmente.

CÓDIGO:

240506549113



PATRÍCIA QUINTÃO

Mestre em Engenharia de Sistemas e computação pela COPPE/UFRJ, Especialista em Gerência de Informática e Bacharel em Informática pela UFV. Atualmente é professora no Gran Cursos Online; Analista Legislativo (Área de Governança de TI), na Assembleia Legislativa de MG; Escritora e Personal & Professional Coach. Atua como professora de Cursinhos e Faculdades, na área de Tecnologia da Informação, desde 2008. É membro: da Sociedade Brasileira de Coaching, do PMI, da ISACA, da Comissão de Estudo de Técnicas de Segurança (CE-21:027.00) da ABNT, responsável pela elaboração das normas brasileiras sobre gestão da Segurança da Informação. Autora dos livros: Informática FCC - Questões comentadas e organizadas por assunto, 3ª. edição e 1001 questões comentadas de informática (Cespe/UnB), 2ª. edição, pela Editora Gen/Método. Foi aprovada nos seguintes concursos: Analista Legislativo, na especialidade de Administração de Rede, na Assembleia Legislativa do Estado de MG; Professora titular do Departamento de Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia; Professora substituta do DCC da UFJF; Analista de TI/Suporte, PRODABEL; Analista do Ministério Público MG; Analista de Sistemas, DATAPREV, Segurança da Informação; Analista de Sistemas, INFRAERO; Analista - TIC, PRODEMGE; Analista de Sistemas, Prefeitura de Juiz de Fora; Analista de Sistemas, SERPRO; Analista Judiciário (Informática), TRF 2ª Região RJ/ES, etc.

GRAN
CONCURSOS

SUMÁRIO

Apresentação	4
Processo Ágil, Scrum, Kanban, Lean	5
Metodologias Ágeis de Desenvolvimento	5
Lean Manufacturing ou Manufatura Enxuta (Pensamento Enxuto – PE ou Sistema Enxuto)	9
Scrum	11
Conceitos Básicos	11
Papéis do Scrum (ou Time Scrum)	13
Artefatos do Scrum	13
Fluxo de Trabalho no Scrum	16
Eventos	16
Kanban	22
Conceitos Básicos	22
Fluxo de Trabalho no Kanban	24
Scrumban	24
Conceitos Básicos	24
Resumo	27
Questões Comentadas em Aula	30
Questões de Concurso	32
Gabarito	37
Gabarito Comentado	38
Referências	51

APRESENTAÇÃO

Olá, querido (a) amigo (a), meus cumprimentos!

“Se seus sonhos estiverem nas nuvens, não se preocupe, pois eles estão no lugar certo; agora, construa os alicerces!”

Os alicerces para uma excelente prova são: persistência, garra, força de vontade, estudo disciplinado e fé em Deus!

Nesta aula merecem destaque os seguintes tópicos: Processo Ágil, Scrum, Kanban e *Lean Development*.

Em caso de dúvidas, acesse o fórum do curso ou entre em contato.

Um abraço,

PROCESSO ÁGIL, SCRUM, KANBAN, LEAN

METODOLOGIAS ÁGEIS DE DESENVOLVIMENTO

A Engenharia de Software vem há anos criando técnicas de modelagem, projeto e desenvolvimento de sistemas. Dentre os desafios dos pesquisadores da área, pode-se citar a preocupação em desenvolver softwares com qualidade garantida, no prazo estabelecido e sem alocar recursos além do previsto. No entanto, muitas das metodologias criadas são consumidoras de recursos, aplicando-se principalmente a grandes sistemas.

Como a Engenharia de Software ainda está evoluindo, novos modelos estão sendo apresentados, como: **desenvolvimento ágil**; dentre outros.

Aqui merecem destaque as **metodologias ágeis de desenvolvimento**, como **Lean**, **SCRUM**, **Kanban** e **Scrumban**.

O desenvolvimento ágil de software fundamenta-se no **Manifesto Ágil** (<https://agilemanifesto.org/>). Por definição, os **12 princípios para a prática do desenvolvimento ágil** são:

Satisfação do cliente	A maior prioridade está em satisfazer o cliente por meio da entrega adiantada e contínua de software de valor.
Mudança em favor da vantagem competitiva	Mudanças de requisitos são bem-vindas, mesmo em fases tardias do desenvolvimento. Os processos ágeis tiram vantagem das mudanças visando vantagem competitiva para o cliente. As metodologias tradicionais são preditivas – esse cenário é ilusório.
Prazos curtos	Entregar software em funcionamento com frequência , desde a cada duas semanas até a cada dois meses, com uma preferência por prazos mais curtos.
Trabalho em conjunto	Tanto pessoas relacionadas a negócios como desenvolvedores devem trabalhar em conjunto , diariamente, durante todo o curso do projeto.
Ambientação e suporte	Para construir projetos ao redor de indivíduos motivados, é preciso dar a eles o ambiente e o suporte necessários , confiando que farão seu trabalho.
Falar na cara	O método mais eficiente de transmitir informações tanto externas como internas para um time de desenvolvimento é por meio de uma conversa face a face . As metodologias tradicionais utilizam documentos, diagramas, relatórios, telefonemas para promover a comunicação no projeto.
Funcionalidade	Um software funcional é a medida primária de progresso . As metodologias tradicionais propunham a entrega de artefatos (Exemplo: Documentação) que, em geral, não agregavam valor algum aos clientes, como também uma forma de medir o progresso do projeto.
Ambiente de sustentabilidade	Processos ágeis promovem um ambiente sustentável , com patrocinadores, desenvolvedores e usuários sendo capazes de manter passos constantes.

Satisfação do cliente	A maior prioridade está em satisfazer o cliente por meio da entrega adiantada e contínua de software de valor.
Padrões altos de tecnologia e design	A contínua atenção à excelência técnica e ao bom <i>design</i> aumenta a agilidade.
Simplicidade	Fazer algo simples é dominar a arte de maximizar a quantidade de trabalho que não precisou ser feito. As metodologias tradicionais, algumas vezes, recorriam a implementações desnecessariamente complexas, a planejamentos exageradamente detalhados, etc.
Autonomia	As melhores arquiteturas, os requisitos e os designs emergem de times auto-organizáveis . As metodologias tradicionais geralmente precisam de um gerente de projetos responsável por organizar o trabalho da equipe como um todo, sendo também responsável pela tomada de decisões.
Reflexões para otimizações	Em intervalos regulares, o time reflete em como ficar mais efetivo , então, se ajustam e otimizam seu comportamento de acordo. As metodologias tradicionais muitas vezes são engessadas, dessa forma não revisitam frequentemente seu comportamento.

De acordo com o Manifesto Ágil deve-se valorizar:

- os **indivíduos e interações** mais que processos e ferramentas;
- **software em funcionamento** mais que documentação abrangente;
- **colaboração com o cliente** mais que negociação de contratos;
- **responder a mudanças** mais que seguir um plano.

São **exemplos** de metodologias ágeis:

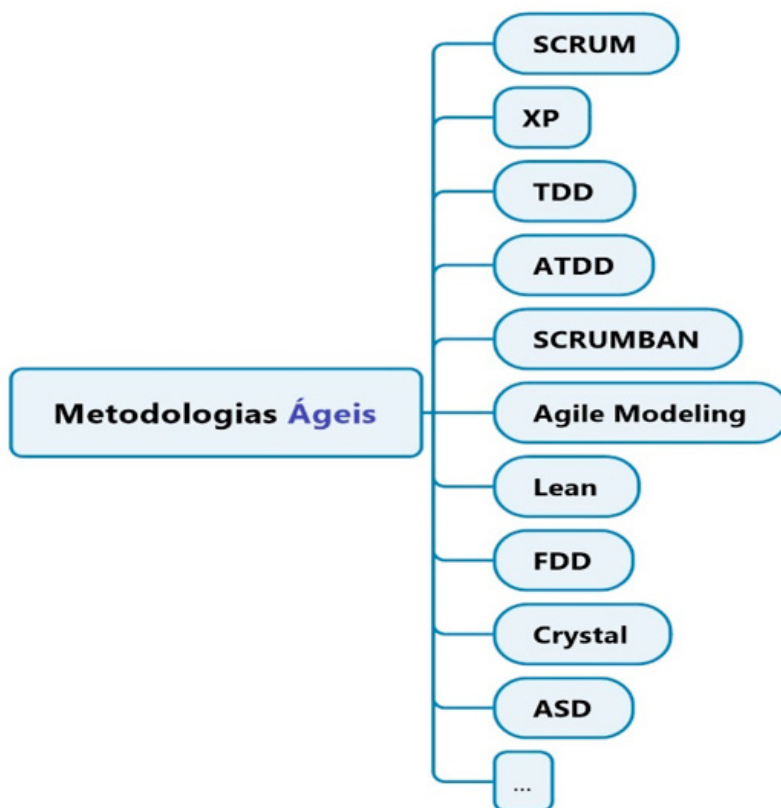


Figura. Metodologias Ágeis. Fonte: Quintão (2021)

Veja a seguir um esquema destacando as Metodologias Ágeis no âmbito dos principais modelos de processo de software.

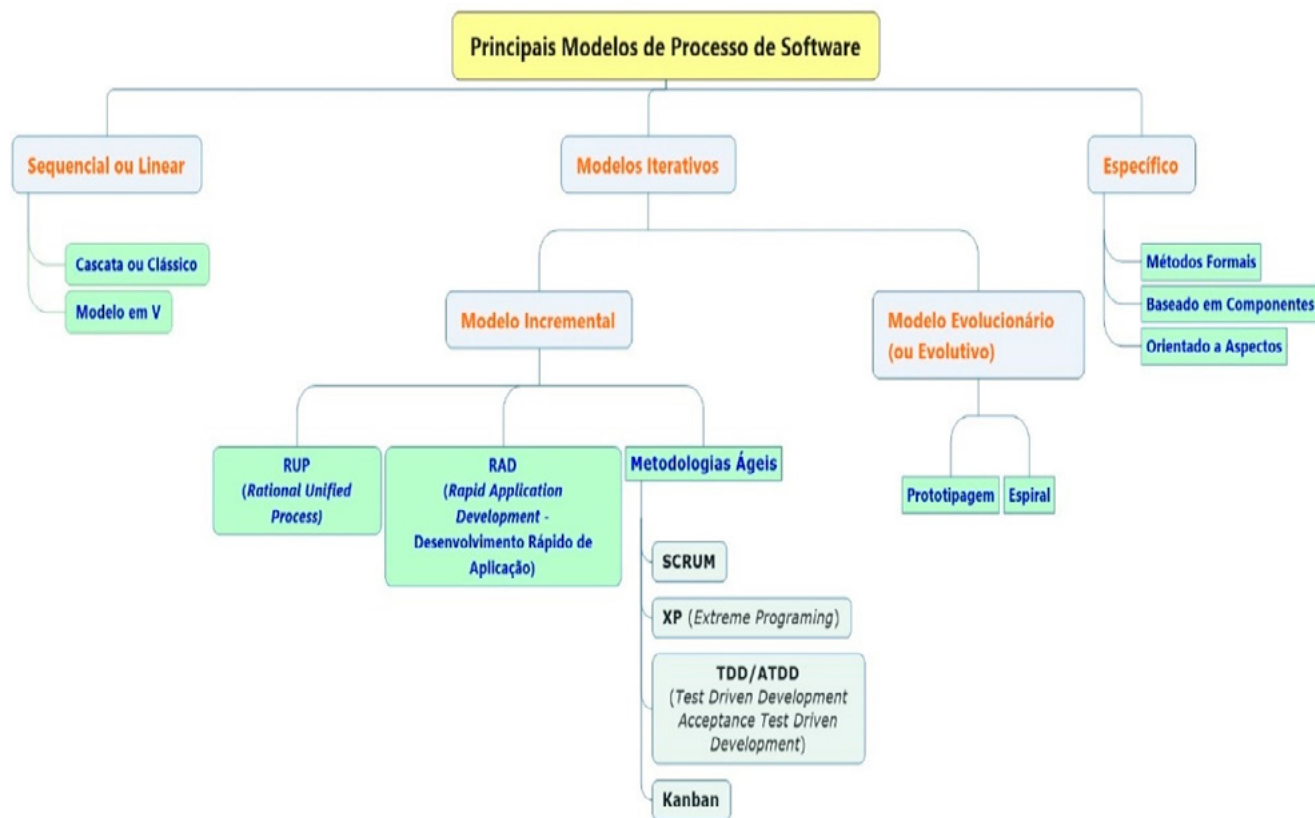


Figura. Principais Modelos de Processo de Software.
Fonte: Quintão (2021)

Veja a seguir algumas características fundamentais entre as metodologias ágeis, segundo Sommerville:

- **não** há especificação detalhada do sistema, e a documentação do projeto é minimizada ou gerada automaticamente;
- o sistema é desenvolvido em uma série de versões;
- interfaces de usuário do sistema são geralmente desenvolvidas com um sistema **iterativo** de desenvolvimento que permite a criação rápida do projeto.

DIRETO DO CONCURSO

001. (ESAF/STN/ANALISTA DE FINANÇAS E CONTROLE/GOVERNANÇA E GESTÃO EM TI/2013)
O desenvolvimento ágil de software fundamenta-se no Manifesto Ágil. Segundo ele deve-se valorizar:

- a) mudança de respostas em vez do seguimento de um plano.
- b) indivíduos e interações em vez de processos e ferramentas.
- c) documentação extensiva operacional em vez de software funcional.
- d) indivíduos e intenções junto a processos e ferramentas.
- e) seguimento de um plano em vez de resposta a mudança.



O Manifesto Ágil contém quatro valores fundamentais, que são:

- Os **indivíduos e suas interações** acima de procedimentos e ferramentas;
- **Software funcional** acima de documentação abrangente;
- A **colaboração dos clientes** acima da negociação de contratos;
- A **capacidade de resposta a mudanças** acima de um plano pré-estabelecido;

Fonte: <http://manifestoagil.com.br/>

Assim, temos:

- a) Errada. Temos aqui a capacidade de resposta às mudanças, ao invés de seguir um plano pré-estabelecido.
- b) Certa. Os **indivíduos e suas interações** acima de procedimentos e ferramentas.
- c) Errada. **Software funcional** acima de documentação abrangente.
- d) Errada. Os **indivíduos e suas interações** acima de procedimentos e ferramentas.
- e) Errada. A **capacidade de resposta a mudanças** acima de um plano pré-estabelecido.

Letra b.

EXEMPLO

A **Agile Alliance Brazil**, ou simplesmente **AAB**, é uma associação sem fins lucrativos criada para **dar apoio administrativo ao evento Agile Brazil** e **fomentar ações para desenvolver a comunidade ágil brasileira**, tornando possíveis iniciativas que consolidem a adoção de metodologias ágeis na Indústria de Software do país.

LEAN MANUFACTURING OU MANUFATURA ENXUTA (PENSAMENTO ENXUTO – PE OU SISTEMA ENXUTO)

Na década de 50, Taiichi Ohno implantou o **Sistema Toyota de Produção (Toyota Production System – TPS)** com o principal **objetivo de identificar e eliminar os possíveis desperdícios**, buscando reduzir custos e aumentar a qualidade e velocidade de entrega do produto aos clientes. Por representar uma maneira de produzir mais utilizando cada vez menos, este sistema foi denominado **Produção enxuta (Lean production ou Lean manufacturing)** por James P. Womack e Daniel T. Jones.

Assim, são vários as traduções e sinônimos para filosofia da **Manufatura Enxuta**, como **Lean Manufacturing, Lean, Pensamento Enxuto (PE)**, até porque não é aplicada apenas na manufatura, ou ainda **Sistema Enxuto**.

Segundo Cruz (2015), **Lean está ligado à eliminação de desperdício em qualquer etapa de um processo**. No desenvolvimento de software, isso está intrinsecamente ligado a funcionalidades que **não** agregam valor para o cliente, à perda de tempo em atividades

que **não** entregam resultado, construção de funcionalidades que **não** serão utilizadas, retrabalho por **baixa qualidade**.

Sua essência é a capacidade de eliminar desperdícios continuamente e resolver problemas de maneira sistemática. Isso implica repensar a maneira como se lidera, gerencia e desenvolve pessoas. É através do pleno engajamento das pessoas envolvidas com o trabalho que se consegue vislumbrar oportunidades de melhoria e ganhos sustentáveis (LEAN, 2017).

Tyagi et al., (2015) destacaram que **as práticas de PE** podem ser aplicadas isoladamente, no entanto, **quando utilizadas de forma organizada e quando apoiada por um trabalho integrado da organização, aumentam potencialmente o desempenho da planta através da implementação de práticas do pensamento enxuto.** A principal razão para esses benefícios é a **sinergia**, o trabalho integrado de todos da organização para reduzir a variabilidade.

O conceito de PE possui aplicação em **toda a organização**, pois criam produtos ou serviços por meio de processos que ultrapassam as fronteiras funcionais com a intenção de criar valor para os clientes que podem ser externos ou internos.

De acordo com Womack e Jones (2004), **existem cinco princípios básicos fundamentais da mentalidade lean: valor, fluxo de valor, fluxo, produção puxada e perfeição.** Vamos à descrição desses princípios:

- **valor:** consiste em definir o que é Valor. Não é a empresa e sim o **cliente** que **define o que é valor**;
- **fluxo de Valor:** consiste **em identificar os processos** que geram valor, aqueles que não geram valor, mas são importantes para a manutenção dos processos e da qualidade e, por fim, aqueles que não agregam valor e que devem ser eliminados;
- **fluxo:** deve-se dar **“fluidez” para os processos e atividades** que restaram;
- **produção puxada:** **conectam-se os processos através de sistemas puxados.**
- **perfeição:** a **busca do aperfeiçoamento contínuo em direção a um estado ideal** deve nortear todos os esforços da empresa (WOMACK e JONES, 2004).

DIRETO DO CONCURSO



002. (FGV/IMBEL/ANALISTA ESPECIALIZADO/ANALISTA ADMINISTRATIVO/REAPLICAÇÃO/2021)

A filosofia “Lean” é uma abordagem com foco em processos e que tem, como essência, a busca pela redução de desperdícios. Assinale a opção que apresenta um dos princípios basilares da filosofia “Lean”.

- a) Melhoria pontual.
- b) Maximização do uso de recursos.
- c) Processamento “empurrado”.
- d) Qualidade perfeita na segunda vez.
- e) Construção de relacionamento de curto prazo com os fornecedores.



O **Lean Manufacturing** é uma filosofia de gestão baseada na **eliminação de desperdícios** para consequente **diminuição de custos, melhoria de qualidade do produto final e maximização do uso de recursos** (WOMACK; JONES; ROOS, 1992).

Letra b.

SCRUM

CONCEITOS BÁSICOS

O **Scrum** é um *framework* de **processo ágil** utilizado para gerenciar e controlar o desenvolvimento de um produto de software através de práticas **iterativas e incrementais**.

A figura seguinte é bem mais interessante para ilustrar a diferença entre incremental e iterativo!

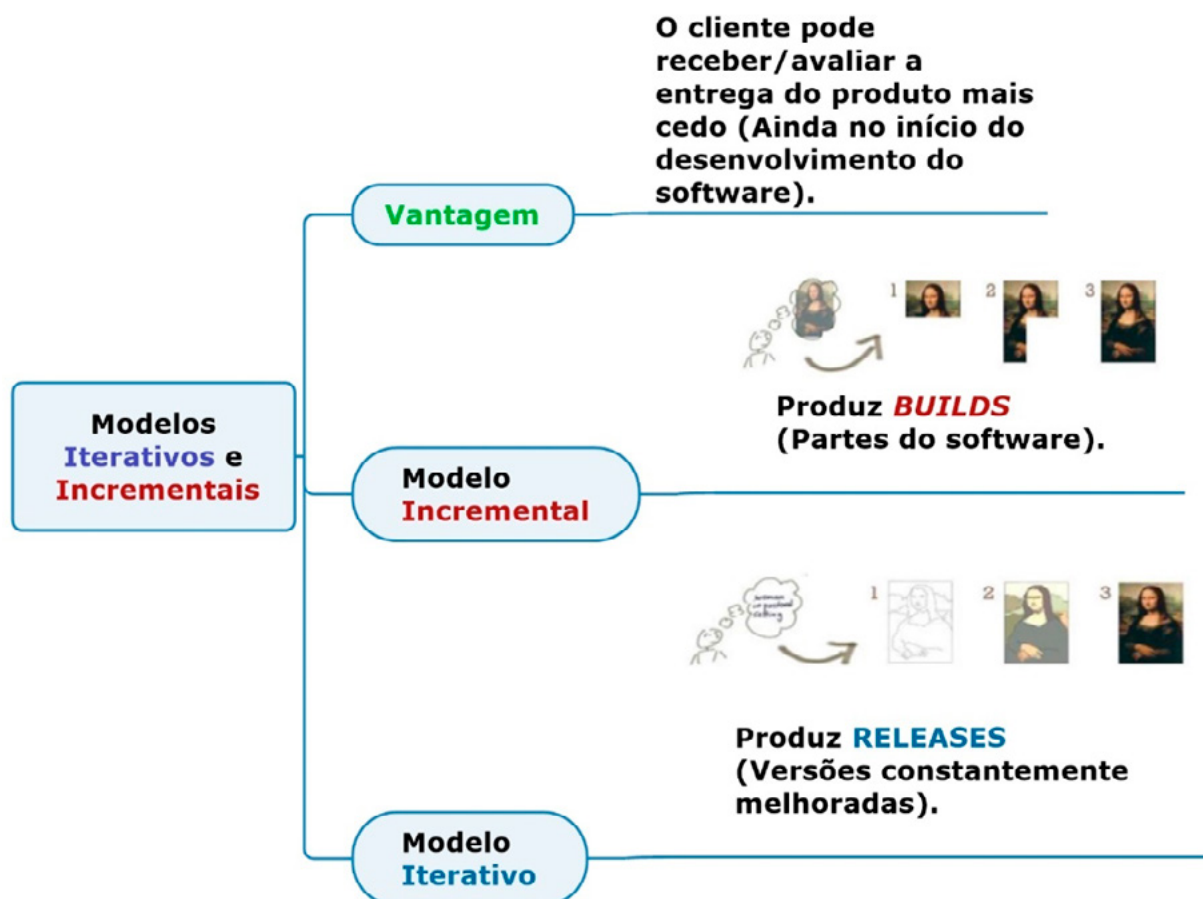


Figura. Modelos Iterativos e Incrementais. Fonte: Quintão (2022)

Scrum é composto por um conjunto de boas práticas de gestão que admite ajustes rápidos, acompanhamento e visibilidade constantes e planos realísticos.

Obs.: Scrum é um processo bastante leve para gerenciar e controlar projetos de desenvolvimento de software e para a criação de produtos.

O **Scrum** é um **processo de desenvolvimento ágil de software** baseado em grupos de práticas e papéis predefinidos.

Ele é um processo **iterativo** e **incremental** para gerenciamento de projetos e desenvolvimento de sistemas, em que cada **sprint** é uma iteração que segue um ciclo PDCA (*Plan, Do, Check, Act*) e entrega um incremento de software pronto.



Figura. Scrum. Fonte: Quintão (2022)

PAPÉIS DO SCRUM (OU TIME SCRUM)

Os principais papéis do Scrum são: **Product Owner (PO)**, **Scrum Master (SM)** e **Development Team (DT)**.

Papéis do Scrum	Descrição
Product Owner (PO)	Dono do Produto. É uma pessoa e não um comitê. Ele pode representar o desejo de um comitê no Product Backlog, mas aqueles que quiserem uma alteração nas prioridades dos itens de backlog devem convencer o Product Owner. É o responsável por maximizar o valor do produto e do trabalho do Time de Desenvolvimento. Como isso é feito pode variar amplamente através das organizações, Times Scrum e indivíduos.
Development Team (DT)	Time de Desenvolvimento. Profissionais que realizam o trabalho de entregar uma versão usável que potencialmente incrementa o produto “pronto” ao final de cada <i>sprint</i> . O Time de Desenvolvimento só responde ao Product Owner. Responsável pela microgestão e pela criação do produto. São auto-organizados . Ninguém (nem mesmo o Scrum Master) diz ao Time de Desenvolvimento como transformar o <i>Product Backlog</i> em incrementos de funcionalidades potencialmente utilizáveis. Individualmente, os integrantes do Time de Desenvolvimento podem ter habilidades especializadas, mas a responsabilidade pertence aos desenvolvedores como um todo.
Scrum Master (SM)	Mestre Scrum. É responsável por garantir que o Scrum seja entendido e aplicado! Ele ajuda a treinar o time de desenvolvimento em autogerenciamento e interdisciplinaridade. O Scrum Master tem o papel de liderança muito importante para o processo. Ele deve remover todo e qualquer obstáculo que surgir durante o desenvolvimento, garantindo que o Scrum Team possa focar no real objetivo definido. Além disso, ele é responsável por fazer com que a equipe siga e pratique o processo e ainda por criar uma atmosfera de ajuda mútua entre a equipe (o resultado é sempre da equipe e não individual).

Não há como fazermos um mapeamento direto entre os papéis do Scrum e os papéis convencionais conhecidos. Não existe a figura única do Gerente de Projetos. Suas responsabilidades estão diluídas entre os papéis citados. Cada um conhece sua participação frente ao projeto e trabalha em conjunto para conseguir alcançar o *goal* definido.

ARTEFATOS DO SCRUM

Segundo o Guia do Scrum, o *framework* possui apenas três artefatos oficiais, que são:

Artefatos Oficiais	Descrição
Product Backlog	Uma lista ordenada (por valor, risco, prioridade, etc.) de requisitos ou funcionalidades que o produto deve conter criada pela Equipe Scrum.

Artefatos Oficiais	Descrição
<i>Sprint Backlog</i>	Conjunto de itens selecionados para serem implementados durante a sprint mais o plano para transformá-los em um incremento.
<i>Product Increment</i>	Ao final de cada sprint, a equipe de desenvolvimento entrega um incremento do produto – resultado do que foi produzido ao final do trabalho realizado na sprint.

Inicialmente um requisito deve fazer parte do **Product Backlog (Produto Backlog)**, que possui descrições de necessidades de clientes de alto nível levantadas.

A tarefa de manter a descrição e refinamento destes requisitos é do **Product Owner**. Ele é responsável por definir, para cada nova *release* de um produto, o objetivo (**Release Goal** ou **Vision**). Ele interage a todo momento com seu cliente final e, dessa forma, conhece suas expectativas e as do mercado.

Esta lista de demandas nunca acaba e de tempos em tempos (a cada nova *release*), um item de **Product backlog** é promovido a **Release backlog** para integrar ao escopo de uma ou mais *Releases*. O **Product Owner** deve comunicar e discutir o novo *goal* com todos os envolvidos em reuniões chamadas de **Release Planning**.

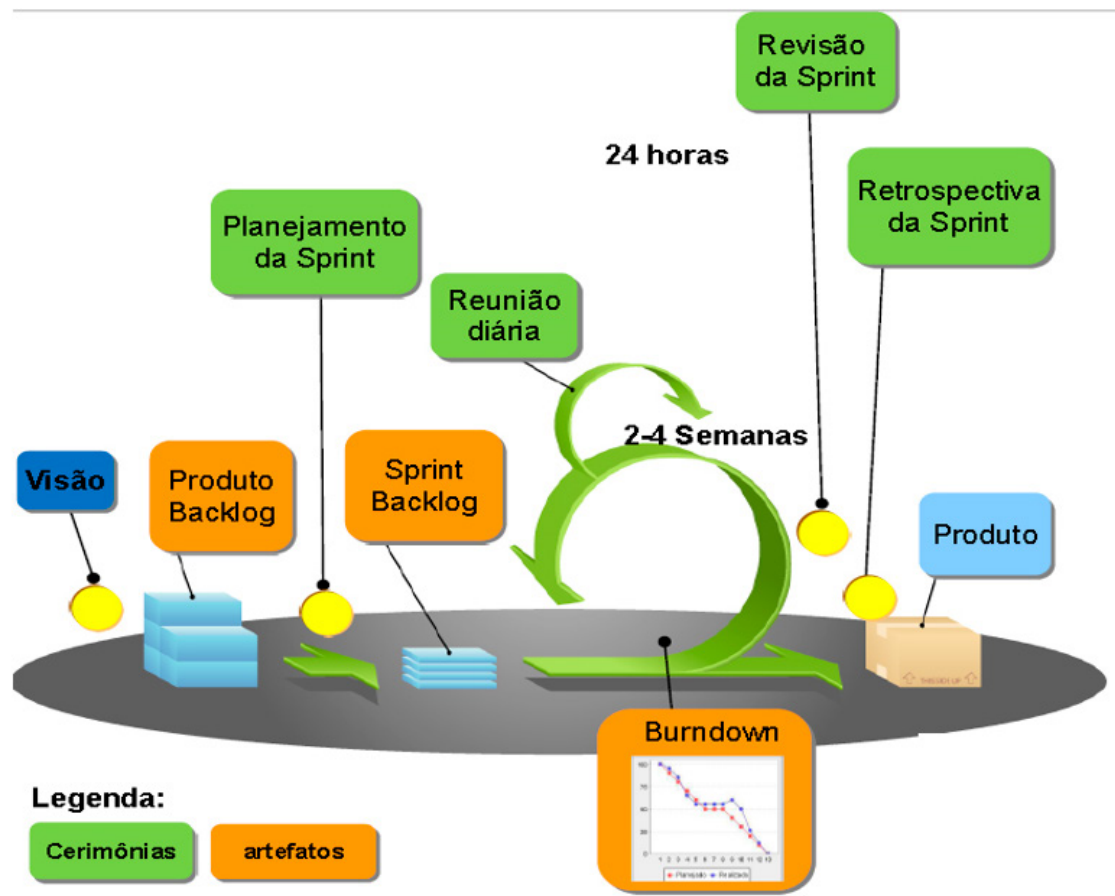


Figura – Estrutura do Scrum

Portanto, a primeira reunião oficial de *Release* que ocorre é a ***Release Planning***. Nessa reunião, são discutidos os recursos envolvidos – tanto computacionais quanto humanos, riscos identificados, prazo acertado e requisitos previstos acordados junto ao cliente e coerentes com a demanda de mercado. Seu propósito principal é definir e comunicar as funcionalidades macro de uma *Release* do produto e fazer com que os envolvidos no projeto entendam e se comprometam com o *Release Goal* definido. As funcionalidades **não** devem ser discutidas em detalhes.

Isso será feito em cada iteração durante reuniões de ***Sprint Planning (Planejamento da Sprint)***. Ela pode ter a presença de diversos papéis: seja cliente externo, desenvolvedor, área comercial, etc. **Cada *release* é composta de iterações curtas, chamadas de *Sprints*.**

Conforme dito, a **primeira reunião é a Release Planning**. Cada Sprint tem reuniões de planejamento, Sprint Planning, que detalham o requisito apresentado durante o Release Planning e ainda possuem seu *Sprint Goal*.

Semelhante ao *goal* do *Release*, tem o mesmo cunho de comunicar a todos os envolvidos o objetivo maior de Sprint corrente. É realizado de forma **iterativa**, através de ciclos de

“tempo fechado” (*Time-Boxed*) em **30 dias corridos**. Ainda sobre a reunião, são conhecidos os requisitos do Release Backlog que serão promovidos como *Selected Backlog*. Eles são discutidos em maiores detalhes e servem para elucidar **O QUE** se espera como resultado final.

Em um Sprint, portanto, são executadas atividades de refinamento de requisitos, análise, projeto, desenvolvimento e testes pelo Scrum Team, devendo resultar em um incremento de produtos funcionando e demonstrável para o *Product Owner* ao final do Sprint, na reunião de ***Sprint Review***.

FLUXO DE TRABALHO NO SCRUM

Conforme destaca Sotile (2018),

no Scrum, o trabalho que será feito no próximo *sprint* é previamente selecionado. A seguir, o *sprint* é bloqueado, o trabalho é feito e depois do *sprint* a fila está vazia.

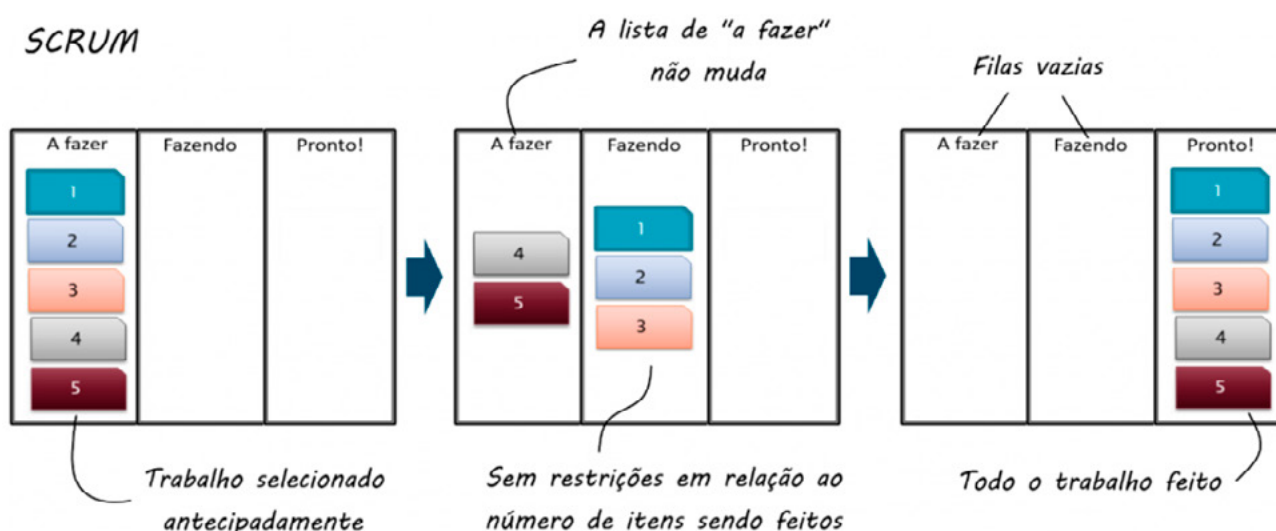


Figura. Trabalho no Scrum. Fonte: Sotile (2018)

EVENTOS

Diariamente são executadas reuniões de acompanhamento e monitoramento do Sprint através da reunião de **Daily Scrum**. Esta reunião deve ter a presença do Scrum Team e Scrum Master obrigatoriamente.

Deve ter a duração máxima de 15 minutos (é interessante até utilizar um cronômetro para isso) e são permitidas somente 3 perguntas:

- O que você fez hoje?
- O que fará amanhã?
- Que impedimentos surgiram e que atrapalharam sua produtividade?

Outros assuntos deverão ser discutidos em outras reuniões, como **Follow-Up Meetings** para detalhamentos e elucidações relativas aos requisitos e *Reestimate Meetings* para aplicação de técnicas como o *Poker Planning* para estimativa de requisitos.

Há uma reunião de **Sprint Planning 2** que promove o desdobramento do *Selected Backlog* em *Sprint Backlog* nos quais aspectos técnicos são discutidos. É a hora do **COMO** as coisas devem ser feitas. Neste momento, pode-se discutir sobre a arquitetura do produto, reuso de componentes, mudança em interfaces, etc.

Ao final do Sprint, é feita uma reunião final de apresentação do trabalho executado ao *Product Owner* chamada **Sprint Review**. Ele será responsável por validar a apresentação e concluir se o Sprint Goal foi atingido. Após esta, reuniões de *debrief* são executadas, chamadas **Retrospective Meetings**.

Reuniões de retrospectiva levam a reflexão acerca do que passou e quais atitudes devem ser tomadas para o correto atingimento dos objetivos. São levantados **aspectos positivos** (**WWW – what went well**) e **negativos** (**WCBI – what can be improved**) tanto organizacionais quanto relativos a Release.

Todos os envolvidos devem estar presentes e são responsáveis por minimizar todo e qualquer risco ao projeto e também por manter os aspectos positivos levantados.

E o mesmo se repete por todos os **Sprints** planejados, com a diferença de que o último **Sprint** possui uma reunião de Release Review, que tem como guia o **Release Goal** definido. **Nunca é demais lembrar que o planejamento é feito durante toda a Release e, portanto, alterações podem ser feitas.** Constitui em uma boa prática poder refazer o planejamento durante cada **Sprint Planning**, mas durante o **Sprint**, o Scrum Master deve procurar “blindar” o Scrum Team de grandes modificações.

As estimativas de tamanho de cada funcionalidade/requisito são executadas obrigatoriamente pelo Scrum Team. Dessa forma, todos sabem para onde ir (**goal**) e é desta forma que se obtém o comprometimento da equipe para com o projeto.

A viabilidade de continuidade dos planos é constantemente verificada. A comunicação de um projeto que utiliza o Scrum é mais efetiva e direta, além de informações estarem sempre à tona com a utilização de quadros brancos (**Agile Radiator**).

A figura seguinte exemplifica um *Agile Radiator* monitorando um projeto real. Eles garantem visibilidade do projeto a todos os envolvidos.

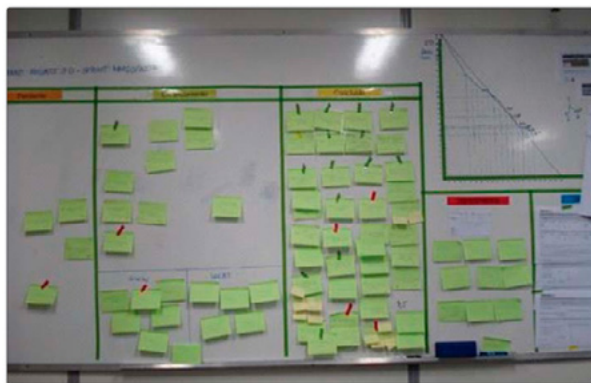


Figura. Agile Radiator

Não há como mascarar o real andamento. O **goal** fica afixado e os requisitos – **através de Post-its** – (***Selected backlogs***) e seus desdobramentos (***Sprint backlogs***) são posicionados na situação em que se encontram (se ainda não iniciados – **planejados**, se sendo executados no momento – **em andamento** e se terminados – 100% **concluídos**). Eles devem ser posicionados de acordo com a prioridade dos mesmos – Business Value declarado pelo *Product Owner*.

Post-its localizados no topo nos dizem ser de maior prioridade que os posicionados no rodapé do quadro branco. Impedimentos (***Impediment Backlog***) que ocorrem durante o Sprint também são evidenciados.

O Scrum tem um gráfico, de nome **BurnDown Chart**, **que é a principal ferramenta de gerenciamento do processo de desenvolvimento de software**. Ele representa o trabalho restante sobre tempo, ou seja, permite visualizar o progresso e/ou a evolução do trabalho executado pela equipe e a quantidade trabalho x tempo que ainda faltam para completar a Sprint. A atualização do **Burndown** deve ser diária, isto facilita a tomada de decisão, pois, poderemos decidir em melhorar a produtividade da equipe e/ou para mitigar risco da Sprint.

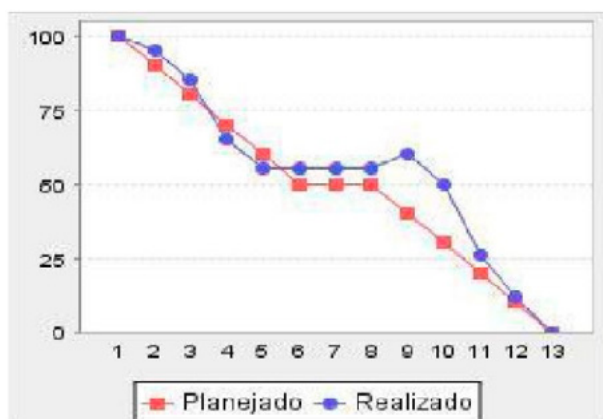


Figura. BurnDown Chart

O Scrum Team diariamente, através de reuniões chamadas Daily Scrum, atualiza o *Agile Radiator* com a situação atual, movendo **post-its** e indicando o trabalho executado durante o período.

Vale destacar que a granularidade de cada **Sprint Backlog** deve ser pequena.

Sua estimativa não deve ultrapassar 8 horas de trabalho e isso se constitui em uma boa prática no gerenciamento do andamento das atividades. Através desta reunião, o time consegue avaliar se é necessário mudar o plano para acertar o rumo, se deve priorizar alguma outra atividade, se muitos impedimentos estão ocorrendo e como minimizá-los, se há algum recurso que está necessitando de ajuda, etc. É real! Isso tudo sempre à luz do **goal** definido. Além de todas estas vantagens, ainda proporciona visibilidade a todos os envolvidos.



Durante cada reunião de **Daily Scrum**, *post-its* são "movimentados" pelo Scrum Team contemplando cada uma das perguntas associadas a esta reunião. Atividades executadas (o que foi feito hoje) são movidas para a seção "**Concluídos**". Atividades planejadas (o que será feito amanhã) são colocadas na seção "**Em Andamento**" e impedimentos são registrados (que impedimentos surgiram? Repare a seção em vermelho no rodapé à direita).

Além destes artefatos, o *Burndown Chart* deve ser atualizado ao final da reunião contabilizando o tamanho total entregue pelo Scrum Team no dia a ser representado. O *Burndown* possui duas retas: uma que exhibe o planejado e outra que reflete o realizado. Ele está registrado no topo à esquerda. Ele contém o total de trabalho restante e só de "batermos o olho" no gráfico conseguimos verificar a evolução do trabalho. Todo o *Agile Radiator* tem esta característica. Se estivermos vendo muitos impedimentos, temos que verificar o que está sendo feito para prevenir novos e se estes estão sendo corretamente

removidos. Se atividades de maior prioridade estão sendo deixadas para trás, a própria equipe pode verificar o que está havendo. O que foi bom (WWW) e o que pode ser melhorado (WCBI) também ficam afixados no quadro branco.

A cada novo Sprint, o Agile Radiator deve ser modificado para retratar a situação atual novamente. E este ciclo se inicia: **planejamento, execução, controle e avaliação do que foi feito para cada nova Release necessária.**

É necessária muita disciplina para seguir esta abordagem. Seus princípios e valores são baseados em dedicação e bom senso de todos os envolvidos.

Suas **práticas** pregam inspeções constantes – para o **feedback rápido** – e aceitação das mudanças e adaptações que o processo deva passar. O comprometimento entre todos os envolvidos também se constitui como um grande diferencial do *framework*. O processo de gestão está permeado entre os diversos papéis existentes.

DIRETO DO CONCURSO

003. (FCC/TJ-AP/ANALISTA JUDICIÁRIO/ÁREA APOIO ESPECIALIZADO/TECNOLOGIA DA INFORMAÇÃO/DESENVOLVIMENTO DE SISTEMAS/2014) O Scrum é um framework ágil para suportar o desenvolvimento e manutenção de produtos complexos. Nesta abordagem,

a) a Equipe de Desenvolvimento é composta pelo Product Owner, o Time Scrum e o Scrum Master. A Equipe é auto-organizável e escolhe qual a melhor forma para completar seu trabalho.

b) o Scrum Master é o responsável por maximizar o valor do produto e do trabalho do Time Scrum. É a única pessoa responsável por gerenciar o Backlog do Produto.

c) o tamanho ideal do Time Scrum, responsável pela codificação dos testes, é de 3 componentes. Se houver mais de 5 integrantes é exigida muita coordenação e não vai funcionar.

d) o trabalho a ser realizado na Sprint é planejado na reunião de planejamento da Sprint, que é um time-box de 8 horas para uma Sprint de 1 mês de duração. Para Sprints menores, este evento deve ser proporcionalmente menor.

e) a Revisão da Sprint é uma oportunidade para o Time Scrum inspecionar a si próprio e criar um plano para melhorias a serem aplicadas na próxima Sprint. Esta é uma reunião *time-boxed* de 5 horas para uma Sprint de 2 meses.



O Scrum é um processo de desenvolvimento de software iterativo e incremental. É um processo ágil que visa gerenciar projetos de difícil planejamento.

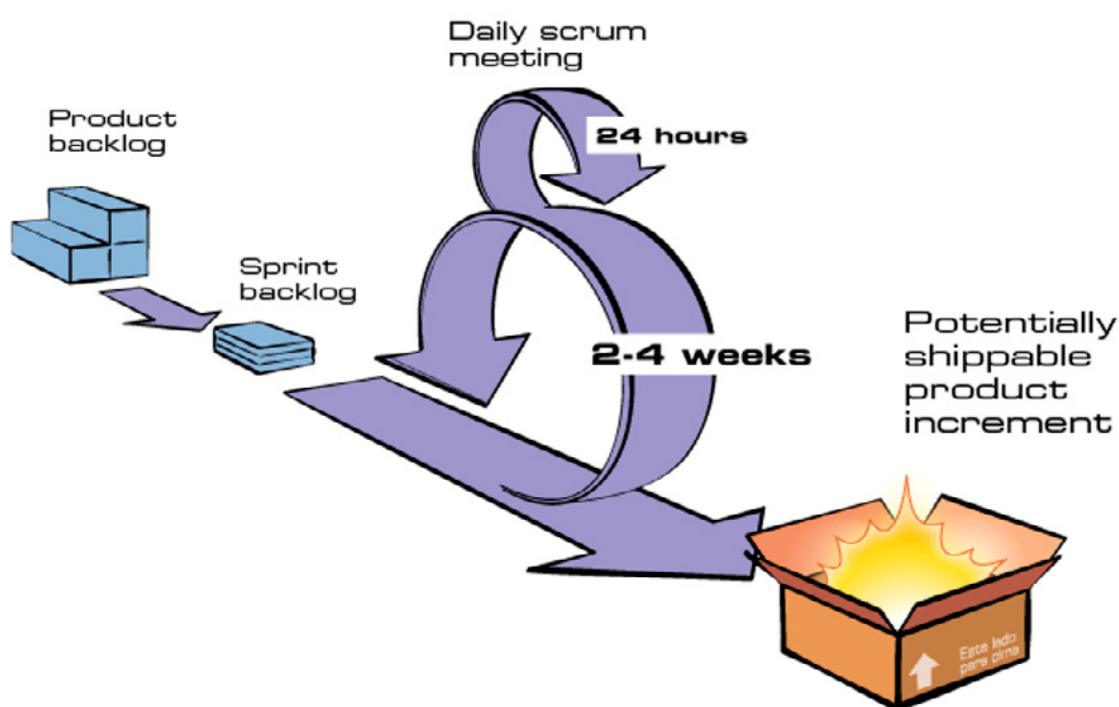
O Scrum não descreve as atividades que devem ser realizadas em cada situação. Ao invés disto, ele contém um conjunto de valores, princípios e práticas que fornecem a base para que uma empresa implemente suas atividades de engenharia e gestão.

Segundo <http://www.desenvolvimentoagil.com.br/scrum/>, no processo de desenvolvimento do Scrum, os projetos são organizados em iterações (Sprints). Um Sprint representa um intervalo no qual um conjunto de tarefas deve ser executado.

As funcionalidades a serem desenvolvidas no sistema são mantidas em uma lista chamada *Product Backlog*. Ao iniciar-se um *Sprint*, realiza-se uma reunião de planejamento (*Sprint Planning Meeting*), em que o *Product Owner* prioriza os itens do *Product Backlog* e a equipe de desenvolvimento identifica as atividades que ela será capaz de implementar durante o *Sprint*. Então, as tarefas alocadas no *Sprint* são transferidas do *Product Backlog* para o *Sprint Backlog*.

Diariamente, em geral pela manhã, a equipe (Scrum Team) realiza uma reunião chamada *Daily Scrum*, visando relatar as tarefas realizadas no dia anterior, identificar impedimentos e priorizar o trabalho do dia que se inicia.

Ao término do Sprint, a equipe de desenvolvimento apresenta as funcionalidades implementadas em uma reunião chamada *Sprint Review Meeting*. Adicionalmente, realiza-se uma *Sprint Retrospective* em que se avalia o trabalho realizado e a equipe parte para o planejamento do próximo Sprint.



Na questão, a única alternativa correta é a letra D, que cita a reunião de planejamento do Scrum (*Sprint Planning Meeting*) em que se definem as tarefas do Sprint, conforme citado anteriormente.

Letra d.

004. (CESPE/ANATEL/ANALISTA ADMINISTRATIVO/ARQUITETURA DE SOLUÇÕES DE TIC/2014)

Acerca dos processos de desenvolvimento de software, julgue o item seguinte:

O Scrum é um conjunto simples e eficaz de regras e ferramentas que são utilizadas para maximizar resultados. O Scrum Master exerce o papel de facilitador e motivador da equipe, além de garantir que as regras e as ferramentas sejam utilizadas com vistas à criatividade do trabalho e ao retorno do investimento.



Scrum é um *framework* de processo dentro do qual podem ser empregados processos e técnicas variadas. Trata-se de um conjunto simples e eficaz de regras e ferramentas que são utilizadas para maximizar resultados. Pode ser aplicado em qualquer contexto no qual um grupo de pessoas trabalhe junto para atingir algum objetivo.

Scrum Master é o responsável pela aplicação dos valores e práticas do Scrum. Remove obstáculos, facilita resultados, garante a plena funcionalidade e produtividade da equipe. Considerado como um “escudo para interferência externas”.

Certo.

005. (FCC/TRE-CE/ANALISTA JUDICIÁRIO/ANALISTA DE SISTEMAS/2012) No SCRUM, *sprint* é

- a) um representante dos stakeholders e do negócio.
- b) uma lista de requisitos que tipicamente vêm do cliente.
- c) uma lista de itens priorizados a serem desenvolvidos para um software.
- d) uma iteração que segue um ciclo (PDCA) e entrega incremento de software pronto.
- e) um conjunto de requisitos, priorizado pelo *Product Owner*.



O Scrum é um processo de desenvolvimento ágil de software baseado em grupos de práticas e papéis predefinidos. Ele é um processo **iterativo e incremental para gerenciamento de projetos e desenvolvimento de sistemas**, em que cada **sprint** é uma iteração que segue um ciclo PDCA (*Plan, Do, Check, Act*) e entrega um incremento de software pronto.

Letra d.

KANBAN

CONCEITOS BÁSICOS

Kanban é um método de desenvolvimento de software com fortes bases em práticas enxutas, e tem como objetivo otimizar o processo de desenvolvimento de software pré-existente.

Henrik destaca as seguintes **práticas** do Kanban:

- 1 - Visualizar o fluxo de trabalho (workflow);**
- 2 - Limitar o trabalho em progresso;**
- 3 - Gerenciar e medir o fluxo.**

Segundo o autor, embora poucas, quando bem utilizadas essas práticas podem levar um time de desenvolvimento de software a ótimos resultados. David Anderson recomenda que se acrescente mais duas condições a essa lista de Kniberg, para se ter um processo ainda mais otimizado:

- 1 - Tornar políticas do processo explícitas;
- 2 - Usar modelos para avaliar Oportunidades de Melhoria.

Entende-se por fluxo a progressão visual dos itens de trabalho através do sistema Kanban.

Assim, uma de suas práticas é o **Gerenciamento do Fluxo**. Nesse contexto:

- visualize o fluxo de trabalho: divida o trabalho em partes, e use colunas nomeadas para ilustrar onde cada item se encontra no fluxo de trabalho;

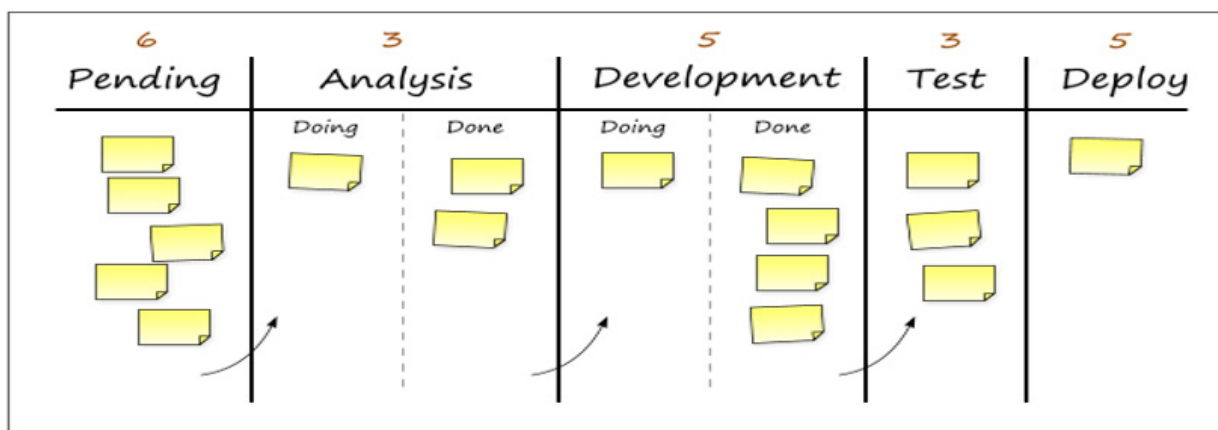


Figura. Quadro Kanban (Fonte: <http://www.kanbanblog.com/explained/>)

- limite o trabalho em progresso (*work in progress*), associando limites explícitos para quantos itens podem estar em progresso em cada estado do fluxo de trabalho;
- acompanhe o tempo de execução da tarefa e otimize o processo para tornar o tempo de execução o menor e mais previsível possível.

Segundo destaca Gomes (2013),

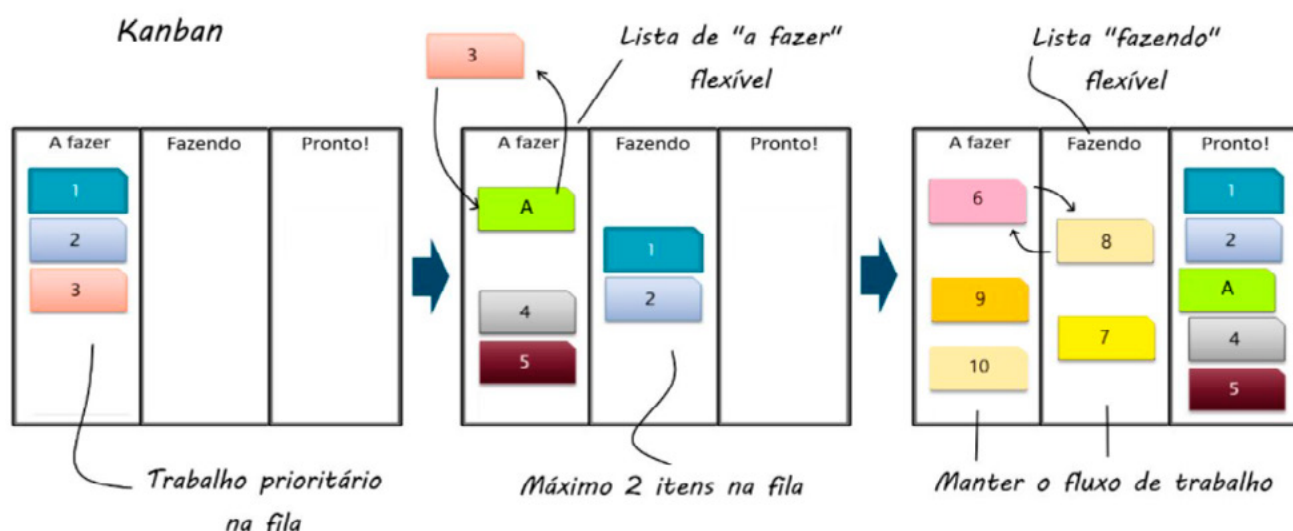
sempre que um item de trabalho parecer estar parado por muito tempo, sem fluir no processo, a equipe deve conversar a respeito para identificar a causa do problema, e então pensar em como melhorar a fluidez do processo. Entre as causas possíveis para **problemas** que podem levar um item de trabalho a não fluir no Kanban podemos citar: dificuldades técnicas, cliente ou analistas

de negócio indisponíveis para esclarecer dúvidas de negócio da equipe, requisitos pouco claros, ambiente de homologação indisponível, longas esperas por informações, etc.

FLUXO DE TRABALHO NO KANBAN

Conforme destaca Sotile (2018),

no Kanban, o **tamanho das filas**, chamado de limite de trabalho em processo, é **limitado**. Isso significa que os itens nas filas podem ser alterados a qualquer momento e que não há “final da sprint”. O trabalho continua fluindo.



Fonte: Sotile (2018)

SCRUMBAN

CONCEITOS BÁSICOS

- **Scrumban (= Scrum + Kanban).**
- É baseado em sistema puxado, no qual a equipe já não planeja o trabalho durante a *sprint planning*, em vez disso, o refinamento é feito continuamente (SOTILE, 2018).
- Usa a natureza prescritiva do Scrum para ser ágil (SOTILE, 2018).
- Usa a melhoria de processo do Kanban para permitir que a equipe melhore continuamente seu processo (SOTILE, 2018).
- Usando o sistema puxado do Kanban o fluxo se torna mais suave à medida que a capacidade de processo melhora (SOTILE, 2018).
- O lead time médio e o tempo de ciclo se tornam o foco principal do desempenho. Se o tempo de ciclo estiver sob controle e a capacidade da equipe for equilibrada com a

demanda, o tempo de espera também estará sob controle. Se o tempo de ciclo estiver sob controle, as *burndowns* são previsíveis e deixam de importar (SOTILE, 2018).

- Conforme destaca Sotile (2018), como a equipe agora coloca o trabalho em uma fila pequena e pronta antes de inseri-lo no trabalho em andamento, o *backlog* da iteração sempre contém algo para ser feito (fluxo ininterrupto).



Figura. Backlog (SOTILE, 2018)

- Em vez de se preocupar em estimar um escopo de trabalho para cada iteração, limita-se o tamanho do *backlog* de iteração **usando-se um tamanho fixo para o backlog**, o qual será executado até zero antes do término do intervalo de planejamento.
- Priorização sob demanda – o processo ideal de planejamento de trabalho deve sempre fornecer à equipe a melhor coisa para trabalhar a seguir, nem mais nem menos
- No Scrumban, podemos fazer o **planejamento de iteração** em intervalos regulares, sincronizados com revisão e retrospectiva, sendo que o objetivo do planejamento é preencher os espaços disponíveis e não preencher todos os espaços. Isso reduz muito a sobrecarga e a cerimônia do planejamento de iteração.

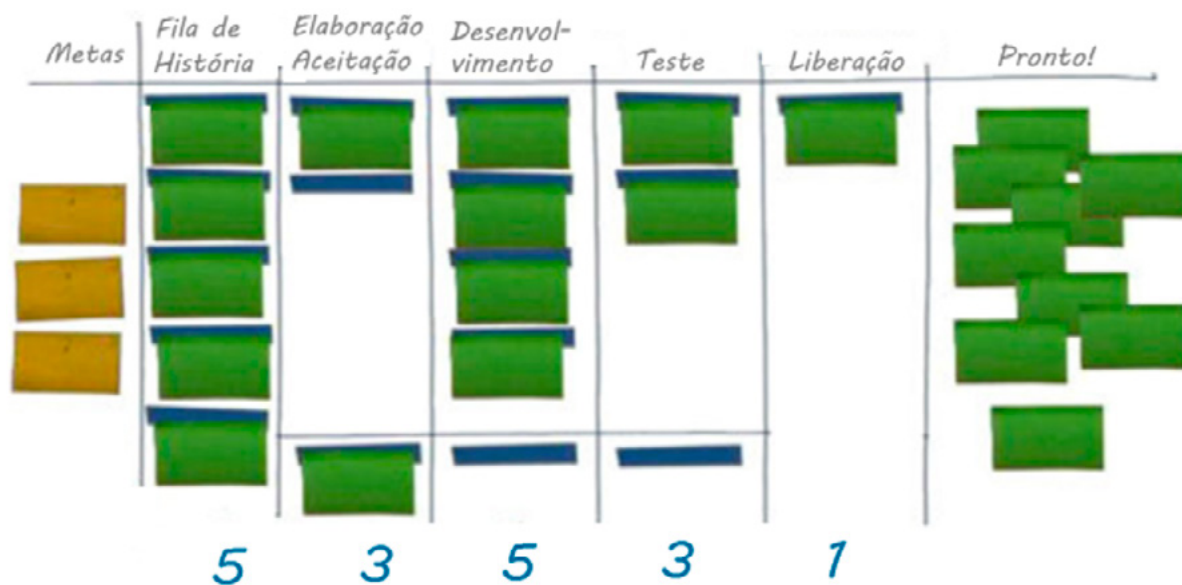


Figura. Exemplo de Scrumban Board (SOTILE, 2018)

Quando Usar Scrumban, segundo Sotile (2018):

- trabalho orientado a eventos, como *help desk* / suporte ou na fase de embalagem;
- projetos com histórias de usuários frequentes e inesperadas ou erros de programação;
- equipes focadas no desenvolvimento de novos produtos:
 - trabalho que antecede o desenvolvimento do *sprint* (*backlog*, pesquisa e desenvolvimento);
 - trabalho após o desenvolvimento do *sprint* (teste, empacotamento e implantação do sistema);
- se o Scrum apresentar problemas de fluxo de trabalho, recursos e processos;
- gerenciar comunidades de melhoria durante / após o início do uso do Scrum.
- **Algumas vantagens do Scrumban**, citadas em Sotile (2018):
 - *Just-in-time* (decisões e fatos justamente quando são necessários);
 - Prazo curto de entrega;
 - Kaizen (melhoria contínua);
 - Minimização de desperdício (tudo o que não agrega valor ao cliente);
 - Melhoria de processo, adicionando alguns valores do Scrum como e quando necessário.

RESUMO

LEAN

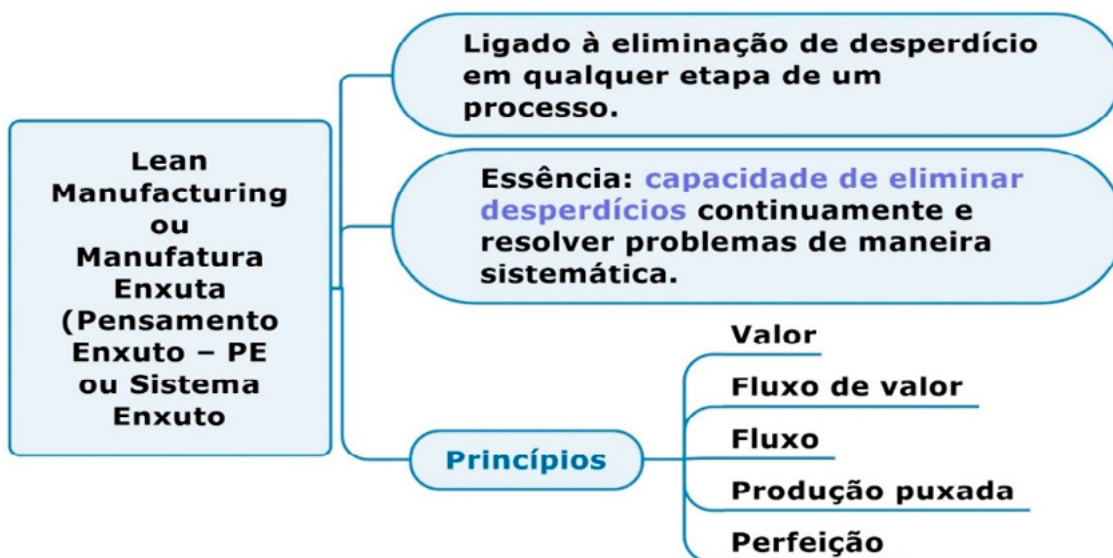


Figura. Lean (QUINTÃO, 2022)

SCRUM

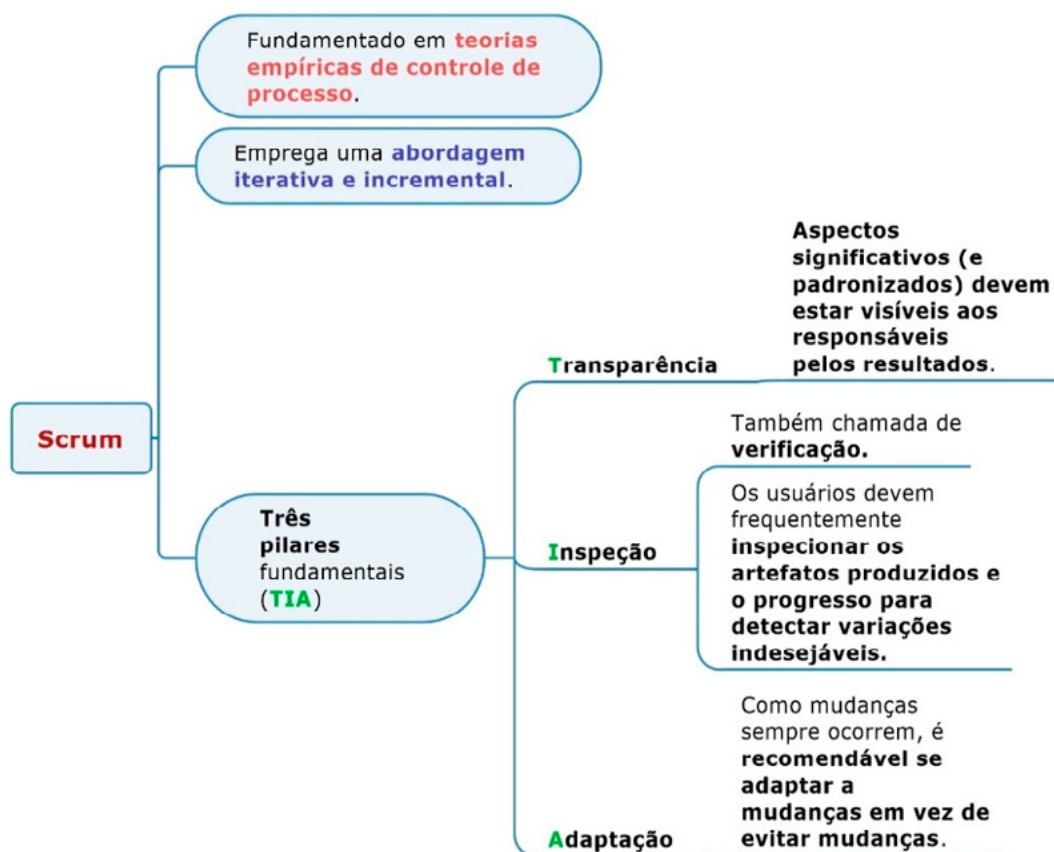


Figura. Scrum (QUINTÃO, 2022)

Papéis do Scrum	
Product Owner (PO)	Dono do Produto. É uma pessoa e não um comitê. Ele pode representar o desejo de um comitê no Product Backlog, mas aqueles que quiserem uma alteração nas prioridades dos itens de backlog devem convencer o Product Owner.
Scrum Master (SM)	Mestre Scrum. É responsável por garantir que o Scrum seja entendido e aplicado! Ele ajuda a treinar o time de desenvolvimento em autogerenciamento e interdisciplinaridade.
Development Team (Dt)	Equipe de Desenvolvimento. Profissionais que realizam o trabalho de entregar uma versão usável que potencialmente incrementa o produto "pronto" ao final de cada sprint.

O framework possui apenas três artefatos oficiais, que são:

Artefatos Oficiais	Descrição
Product Backlog	Uma lista ordenada (por valor, risco, prioridade, etc.) de requisitos ou funcionalidades que o produto deve conter criada pela Equipe Scrum.

Artefatos Oficiais	Descrição
Sprint Backlog	Conjunto de itens selecionados para serem implementados durante a sprint mais o plano para transformá-los em um incremento.
Product Increment	Ao final de cada sprint, a equipe de desenvolvimento entrega um incremento do produto – resultado do que foi produzido ao final do trabalho realizado na sprint.

SCRUM X SCRUMBAN

	Scrum	Scrumban
Artefatos	Conselho, <i>backlogs</i> , burndowns	Somente conselho
Cerimônias	Scrum diário, planejamento da <i>sprint</i> , revisão da <i>sprint</i> , retrospectiva da <i>sprint</i>	Scrum diário (planejamento, revisão e retrospectiva conforme necessário)
Iterações	Sim (<i>sprints</i>)	Não (fluxo contínuo)
Estimativa	Sim	Não (tamanho similar)
Equipes	Deve ser multifuncional	Pode ser especializado
Papéis	<i>Product Owner</i> , <i>Scrum Master</i> , Equipe	Equipe + papéis necessários
Trabalho de equipe	Colaborativo, conforme as tarefas necessitem	Todos juntos para atingir objetivos
Trabalho em andamento	Controlado pelo conteúdo da <i>sprint</i>	Controlado pelo estado do fluxo de trabalho
Mudanças	Devem esperar pela próxima <i>sprint</i>	Adicionadas ao quadro conforme necessário
Backlog do produto	Lista de histórias priorizadas e estimadas	Cartões <i>just-in-time</i>
Impedimentos	Tratados imediatamente	Evitados

Fonte: Sotile (2018)

QUESTÕES COMENTADAS EM AULA

001. (ESAF/STN/ANALISTA DE FINANÇAS E CONTROLE/GOVERNANÇA E GESTÃO EM TI/2013)

O desenvolvimento ágil de software fundamenta-se no Manifesto Ágil. Segundo ele deve-se valorizar:

- a) mudança de respostas em vez do seguimento de um plano.
- b) indivíduos e interações em vez de processos e ferramentas.
- c) documentação extensiva operacional em vez de software funcional.
- d) indivíduos e intenções junto a processos e ferramentas.
- e) seguimento de um plano em vez de resposta a mudança.

002. (FGV/IMBEL/ANALISTA ESPECIALIZADO/ANALISTA ADMINISTRATIVO/REAPLICAÇÃO/2021)

A filosofia “Lean” é uma abordagem com foco em processos e que tem, como essência, a busca pela redução de desperdícios. Assinale a opção que apresenta um dos princípios basilares da filosofia “Lean”.

- a) Melhoria pontual.
- b) Maximização do uso de recursos.
- c) Processamento “empurrado”.
- d) Qualidade perfeita na segunda vez.
- e) Construção de relacionamento de curto prazo com os fornecedores.

003. (FCC/TJ-AP/ANALISTA JUDICIÁRIO/ÁREA APOIO ESPECIALIZADO/TECNOLOGIA DA INFORMAÇÃO/DESENVOLVIMENTO DE SISTEMAS/2014) O Scrum é um framework ágil para suportar o desenvolvimento e manutenção de produtos complexos. Nesta abordagem,

a) a Equipe de Desenvolvimento é composta pelo *Product Owner*, o *Time Scrum* e o *Scrum Master*. A Equipe é auto-organizável e escolhe qual a melhor forma para completar seu trabalho.

b) o Scrum Master é o responsável por maximizar o valor do produto e do trabalho do Time Scrum. É a única pessoa responsável por gerenciar o Backlog do Produto.

c) o tamanho ideal do Time Scrum, responsável pela codificação dos testes, é de 3 componentes. Se houver mais de 5 integrantes é exigida muita coordenação e não vai funcionar.

d) o trabalho a ser realizado na Sprint é planejado na reunião de planejamento da Sprint, que é um time-box de 8 horas para uma Sprint de 1 mês de duração. Para Sprints menores, este evento deve ser proporcionalmente menor.

e) a Revisão da Sprint é uma oportunidade para o Time Scrum inspecionar a si próprio e criar um plano para melhorias a serem aplicadas na próxima Sprint. Esta é uma reunião *time-boxed* de 5 horas para uma Sprint de 2 meses.

004. (CESPE/ANATEL/ANALISTA ADMINISTRATIVO/ARQUITETURA DE SOLUÇÕES DE TIC/2014)

Acerca dos processos de desenvolvimento de software, julgue o item seguinte:

O Scrum é um conjunto simples e eficaz de regras e ferramentas que são utilizadas para maximizar resultados. O Scrum Master exerce o papel de facilitador e motivador da equipe, além de garantir que as regras e as ferramentas sejam utilizadas com vistas à criatividade do trabalho e ao retorno do investimento.

005. (FCC/TRE-CE/ANALISTA JUDICIÁRIO/ANALISTA DE SISTEMAS/2012) No SCRUM, *sprint* é

- a) um representante dos stakeholders e do negócio.
- b) uma lista de requisitos que tipicamente vêm do cliente.
- c) uma lista de itens priorizados a serem desenvolvidos para um software.
- d) uma iteração que segue um ciclo (PDCA) e entrega incremento de software pronto.
- e) um conjunto de requisitos, priorizado pelo *Product Owner*.

QUESTÕES DE CONCURSO

006. (FGV/TJ-DFT/ANALISTA JUDICIÁRIO/ANÁLISE DE SISTEMAS/2022) Uma equipe de analista de sistemas está desenvolvendo o software ProgramaTJ aplicando a metodologia Lean. A equipe decidiu implementar apenas as funcionalidades formalmente requisitadas pelo cliente, evitando adicionar qualquer funcionalidade extra à ProgramaTJ por conta própria. Essa decisão da equipe remete, de forma direta, ao princípio da metodologia Lean para o desenvolvimento de software de:

- a) otimização do todo;
- b) adiar comprometimento;
- c) eliminação de desperdícios;
- d) respeitar as pessoas;
- e) criação de conhecimento.

007. (UFES/UFES/ENGENHEIRO/PRODUÇÃO/2017) A Produção Enxuta (Lean Production)

- a) é direcionada a sistemas automatizados que eliminam ou diminuem a necessidade de intensidade de mão de obra.
- b) objetiva eliminar os desperdícios.
- c) serve de referência para a produção mensal, tornando enxuto o estoque necessário.
- d) reduz o número de máquinas em uma linha de produção, recorrendo, para isso, à produção máxima de cada máquina em uso.
- e) objetiva reduzir o número de pessoas envolvidas no processo produtivo.

008. (FGV/ANALISTA/IBGE/ANÁLISE DE PROJETOS/2016) Uma organização adotou a filosofia Lean para a análise de seus processos e para promover a sua melhoria. Ao mapear sua cadeia de valor, ela identificou desperdícios básicos do Lean, como:

- a) produção menor que a necessária;
- b) trabalho em itens prioritários;
- c) tempo para mover coisas dentro de um processo ou entre eles;
- d) movimentação sem planejamento e em layout organizacional ruim;
- e) processamento de grande valor.

009. (CESPE/TCU/2015) O método para a implantação de mudanças denominado Kanban não prevê papéis nem cerimônias específicas.

010. (FCC/ANALISTA JUDICIÁRIO/TST/APOIO ESPECIALIZADO/ANÁLISE DE SISTEMAS/2017) Um Analista de Sistemas do Tribunal Superior do Trabalho – TST, de modo hipotético,

necessitou aplicar princípios ágeis e de controle usando elementos de três modelos, em processos de manutenção de software. Considere:

I – Dividir o cronograma em iterações time-box ou ciclos (sprints).

II – Orientar o trabalho a eventos ao invés de limite de tempo.

III – Aplicar a programação em pares, integração contínua, orientação a testes (TDD.), revisão de código e todas as demais prescrições antes da implantação.

As características acima correspondem, respectivamente, a

- a) Kanban, XP e Scrum.
- b) Kanban, Scrum e XP.
- c) XP, Scrum e Kanban.
- d) Scrum, XP e Kanban.
- e) Scrum, Kanban e XP.

011. (FCC/2014/AL-PE/ANALISTA LEGISLATIVO/SISTEMAS/ENGENHARIA DE SOFTWARE) Scrum e XP são duas metodologias ágeis que provêm práticas e regras que apresentam diferenças e também pontos em comum. Comparando-se estas metodologias, é correto afirmar:

- a) A XP enfatiza a proximidade física do cliente com a equipe de desenvolvimento para facilitar a comunicação. No Scrum existem diversos eventos formais, tais como sprint backlog meeting e product backlog review, que incentivam a comunicação entre todos os profissionais envolvidos no projeto.
- b) As duas metodologias utilizam iterações curtas de desenvolvimento (sprints), mas divergem no tempo de duração das mesmas. Enquanto no Scrum uma sprint dura de 15 minutos a 8 horas, na XP costuma durar de 1 a 24 horas.
- c) Tanto o Scrum quanto a XP explicitamente não permitem que ocorram mudanças de escopo ou definição dentro de uma sprint. Por isso o cliente deve validar todos os requisitos no início do projeto, isso vai contribuir para evitar atrasos e até mesmo construções erradas.
- d) A XP enfatiza que não se deve fazer horas extras constantemente e, se isso ocorrer, existem problemas no projeto que devem ser resolvidos não com aumento de horas, mas com melhor planejamento. O Scrum enfatiza que equipes auto-organizáveis escolhem qual a melhor forma para completarem seu trabalho.
- e) O Scrum estabelece que os testes devem ocorrer o tempo todo durante o desenvolvimento, principalmente usando técnicas automatizadas. Na XP os testes podem ser realizados apenas na parte final de cada sprint, usando a técnica de refatoração, que busca validar todas as funcionalidades, pensando estrategicamente em como refatorar o código que está sendo implementado.

012. (FCC/2014/TRT 13ª REGIÃO/PB/ANALISTA JUDICIÁRIO/TECNOLOGIA DA INFORMAÇÃO)

No *Scrum*, um projeto se inicia com uma visão simples do produto que será desenvolvido. A visão pode ser vaga a princípio e ir tornando-se clara aos poucos. O **I**, então, transforma essa visão em uma lista de requisitos funcionais e não-funcionais para que, quando forem desenvolvidos, reflitam essa visão. Essa lista, chamada de **II**, é priorizada pelo **III** de forma que os itens que gerem maior valor ao produto tenham maior prioridade.

Completa, correta e respectivamente, as lacunas I, II e III:

- a) *Daily Scrum* - *Scrum Team* - *Sprint*
- b) *Daily Scrum* - *Product Backlog* - *Sprint Planning Meeting*
- c) *Product Owner* - *Sprint* - *Product Backlog*
- d) *Scrum Team* - *Sprint Planning Meeting* - *Product Owner*
- e) *Product Owner* - *Product Backlog* - *Product Owner*

013. (CESPE/2013/MPOG/TECNOLOGIA DA INFORMAÇÃO) Com relação às metodologias ágeis de desenvolvimento, julgue os itens a seguir. Do ponto de vista metodológico, o software livre é considerado uma abordagem similar aos métodos ágeis.

014. (CESPE/SERPRO/ANALISTA/DESENVOLVIMENTO DE SISTEMAS/ENGENHARIA DE SOFTWARE/2013) Julgue os itens a seguir, acerca de metodologias ágeis de desenvolvimento. Kanban é um método de desenvolvimento de software que tem como uma de suas práticas o gerenciamento do fluxo de trabalho, que deve ser monitorado, medido e reportado a cada estado do fluxo.

015. (FCC/ASSEMBLEIA LEGISLATIVA DO ESTADO DE SÃO PAULO/AGENTE TÉCNICO LEGISLATIVO/TECNOLOGIA DA INFORMAÇÃO/2010) São consideradas metodologias ágeis de desenvolvimento de software:

- a) XP e UP.
- b) SCRUM e DSDM.
- c) SCRUM e RUP.
- d) DSDM e Cascata.
- e) Cascata e PRINCE2.

016. (CESPE/BANCO DA AMAZÔNIA/TÉCNICO CIENTÍFICO/ÁREA: TECNOLOGIA DA INFORMAÇÃO/ARQUITETURA DE TECNOLOGIA/2010) O Scrum é utilizado, como função primária, para o gerenciamento de projetos de desenvolvimento de software, mas também tem sido usado como *extreme programming* e outras metodologias de desenvolvimento. Teoricamente, o Scrum pode ser aplicado em qualquer contexto no qual um grupo de pessoas necessite trabalhar juntas para atingir um objetivo comum.

017. (FUMARC/CEMIG/ANALISTA DE TI JUNIOR/2010) Sobre modelos de processo de desenvolvimento de software, assinale a alternativa INCORRETA:

- a) O Scrum é um processo de desenvolvimento ágil de software baseado em grupos de práticas e papéis pré-definidos. Ele é um processo iterativo e incremental para gerenciamento de projetos e desenvolvimento de sistemas, onde cada *sprint* é uma iteração que segue um ciclo PDCA (*Plan, Do, Check, Act*) e entrega um incremento de software pronto.
- b) O design centrado no usuário (UCD) é uma abordagem do processo de desenvolvimento de software baseada no entendimento explícito dos usuários, tarefas, e tem como objetivo principal o casamento entre o modelo conceitual embutido no sistema pelo projetista e o modelo mental do usuário.
- c) Programação extrema (XP – *extreme programming*) é um processo de desenvolvimento ágil baseado em *feedback* rápido, e simplicidade; com enfoque explícito em tempo, custo e qualidade no desenvolvimento, que são alcançados através de uma definição rígida do escopo das funcionalidades da aplicação.
- d) O modelo em espiral é um processo de desenvolvimento de software que intercala etapas de projeto e prototipação, combinando conceitos de desenvolvimento *top-down* e *bottom-up*, e permitindo, desta forma, análise de riscos e estimativas do progresso do trabalho mais realistas.

018. (FCC/TRF 4ª REGIÃO/ANALISTA JUDICIÁRIO/TECNOLOGIA DA INFORMAÇÃO/2010) Na fase de desenvolvimento do Scrum, o software é desenvolvido em processos iterativos denominados

- a) *Building Products*.
- b) *Product Backlog*.
- c) Sprint.
- d) *Product Owner*.
- e) *Product Backlog Cycle*.

019. (CEBRASPE/CESPE/ANALISTA DE PREVIDÊNCIA COMPLEMENTAR/FUNPRESP-EXE/TECNOLOGIA/2022) Acerca da engenharia de software, julgue o item que se segue.
Na fase de desenvolvimento do Scrum, os requisitos são escritos no *product backlog*.

020. (UFMT/PERITO OFICIAL/POLITEC MT/CRIMINAL/CIÊNCIA DA COMPUTAÇÃO OU INFORMÁTICA/2022) No SCRUM, o papel responsável por garantir o máximo de valor agregado ao produto é:

- a) do Cliente.
- b) do *Scrum Master*.
- c) da Equipe.

- d) do *Product Owner*.
- e) do Desenvolvedor.

GABARITO

1. b
2. b
3. d
4. c
5. d
6. c
7. b
8. d
9. C
10. e
11. d
12. e
13. C
14. C
15. b
16. C
17. c
18. c
19. E
20. d

GABARITO COMENTADO

006. (FGV/TJ-DFT/ANALISTA JUDICIÁRIO/ANÁLISE DE SISTEMAS/2022) Uma equipe de analista de sistemas está desenvolvendo o software ProgramaTJ aplicando a metodologia Lean. A equipe decidiu implementar apenas as funcionalidades formalmente requisitadas pelo cliente, evitando adicionar qualquer funcionalidade extra à ProgramaTJ por conta própria. Essa decisão da equipe remete, de forma direta, ao princípio da metodologia Lean para o desenvolvimento de software de:

- a) otimização do todo;
- b) adiar comprometimento;
- c) eliminação de desperdícios;
- d) respeitar as pessoas;
- e) criação de conhecimento.



O *Lean Manufacturing* é uma filosofia de gestão baseada na **eliminação de desperdícios, eliminando do processo as atividades desnecessárias e que não agregam valor**, para consequente diminuição de custos e melhoria de qualidade do produto final (WOMACK; JONES; ROOS, 1992).

Letra c.

- 007.** (UFES/UFES/ENGENHEIRO/PRODUÇÃO/2017) A Produção Enxuta (Lean Production)
- a) é direcionada a sistemas automatizados que eliminam ou diminuem a necessidade de intensidade de mão de obra.
 - b) objetiva eliminar os desperdícios.
 - c) serve de referência para a produção mensal, tornando enxuto o estoque necessário.
 - d) reduz o número de máquinas em uma linha de produção, recorrendo, para isso, à produção máxima de cada máquina em uso.
 - e) objetiva reduzir o número de pessoas envolvidas no processo produtivo.

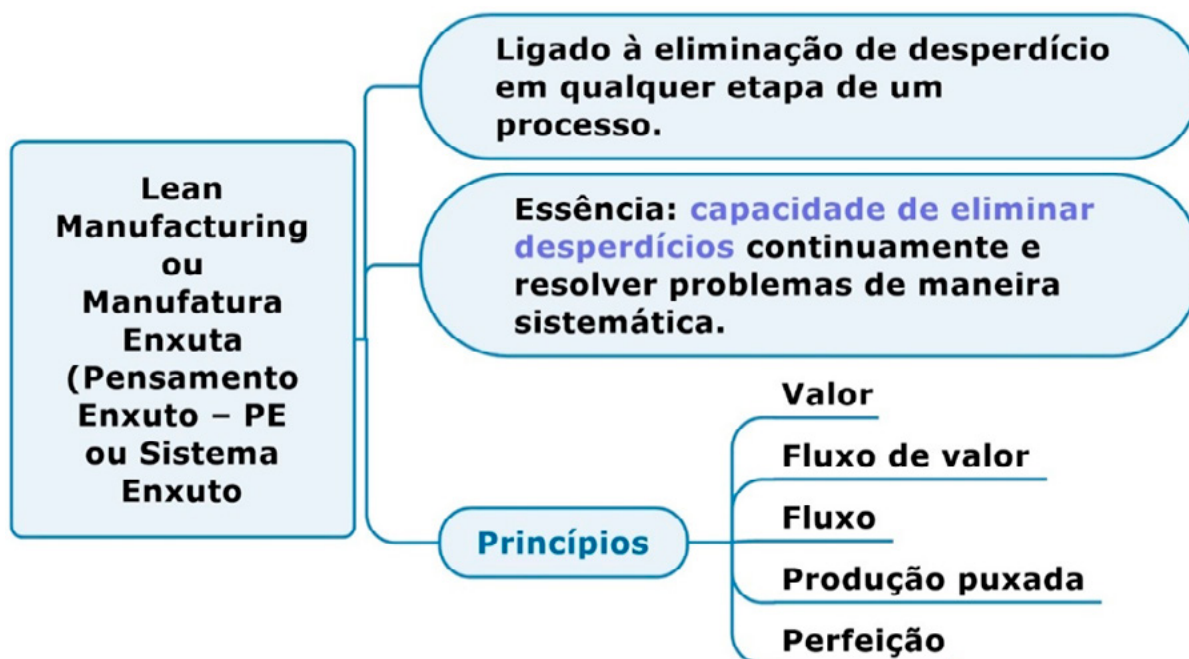


Figura. Lean (QUINTÃO, 2022)

Letra b.

008. (FGV/ANALISTA/IBGE/ANÁLISE DE PROJETOS/2016) Uma organização adotou a filosofia Lean para a análise de seus processos e para promover a sua melhoria. Ao mapear sua cadeia de valor, ela identificou desperdícios básicos do Lean, como:

- a) produção menor que a necessária;
- b) trabalho em itens prioritários;
- c) tempo para mover coisas dentro de um processo ou entre eles;
- d) movimentação sem planejamento e em layout organizacional ruim;
- e) processamento de grande valor.



A filosofia “Lean” é uma abordagem com foco em processos e que tem, como essência, a busca pela **redução de desperdícios, eliminando do processo as atividades desnecessárias e que não agregam valor!**

Ao mapear sua cadeia de valor, a organização identificou que a movimentação **sem** planejamento e em *layout* organizacional **ruim** pode gerar processos desnecessários e, conseqüentemente, desperdícios.

Letra d.

009. (CESPE/TCU/2015) O método para a implantação de mudanças denominado Kanban não prevê papéis nem cerimônias específicas.



O Kanban pode ser considerado como um acelerador para a condução de mudanças ou até mesmo um método para implantação de mudanças em uma organização. Ele não prescreve papéis, práticas ou cerimônias específicas (como acontece, por exemplo, no Scrum). Em vez disso, ele oferece uma série de princípios para otimizar o fluxo e a geração de valor dos sistemas de entrega de software.

Obs.: Kanban é um método de desenvolvimento de software que tem como uma de suas práticas o gerenciamento do fluxo de trabalho, que deve ser monitorado, medido e reportado a cada estado do fluxo.

Certo.

010. (FCC/ANALISTA JUDICIÁRIO/TST/APOIO ESPECIALIZADO/ANÁLISE DE SISTEMAS/2017) Um Analista de Sistemas do Tribunal Superior do Trabalho – TST, de modo hipotético, necessitou aplicar princípios ágeis e de controle usando elementos de três modelos, em processos de manutenção de software. Considere:

I – Dividir o cronograma em iterações time-box ou ciclos (sprints).

II – Orientar o trabalho a eventos ao invés de limite de tempo.

III – Aplicar a programação em pares, integração contínua, orientação a testes (TDD.), revisão de código e todas as demais prescrições antes da implantação.

As características acima correspondem, respectivamente, a

- a) Kanban, XP e Scrum.
- b) Kanban, Scrum e XP.
- c) XP, Scrum e Kanban.
- d) Scrum, XP e Kanban.
- e) Scrum, Kanban e XP.



Atividade	Modelos
I – Dividir o cronograma em iterações time-box ou ciclos (sprints).	No <u>Scrum</u> , os projetos progridem por intermédio de uma série sequencial de sprints. Cada sprint possui um período fixo que normalmente varia de duas a quatro semanas.
II – Orientar o trabalho a eventos ao invés de limite de tempo.	Diferentemente do Scrum que possui <i>sprints</i> regulares de tamanho fixo, o <u>Kanban</u> possui fluxo contínuo, entrega contínua e mudanças podem ocorrer a qualquer momento.

Atividade	Modelos
III – Aplicar a programação em pares, integração contínua, orientação a testes (TDD.), revisão de código e todas as demais prescrições antes da implantação.	Esse item destaca práticas da XP. Na programação em pares duas pessoas trabalham juntas em uma mesma estação de trabalho para codificar. Conforme o trabalho é completado, o código é integrado ao trabalho dos outros (eis a integração contínua!). Ainda, cabe destacar que os programadores só partem para codificação depois que testes de unidade foram criados (TDD).

Letra e.

011. (FCC/2014/AL-PE/ANALISTA LEGISLATIVO/SISTEMAS/ENGENHARIA DE SOFTWARE) Scrum e XP são duas metodologias ágeis que provêm práticas e regras que apresentam diferenças e também pontos em comum. Comparando-se estas metodologias, é correto afirmar:

- A XP enfatiza a proximidade física do cliente com a equipe de desenvolvimento para facilitar a comunicação. No Scrum existem diversos eventos formais, tais como sprint backlog meeting e product backlog review, que incentivam a comunicação entre todos os profissionais envolvidos no projeto.
- As duas metodologias utilizam iterações curtas de desenvolvimento (sprints), mas divergem no tempo de duração das mesmas. Enquanto no Scrum uma sprint dura de 15 minutos a 8 horas, na XP costuma durar de 1 a 24 horas.
- Tanto o Scrum quanto a XP explicitamente não permitem que ocorram mudanças de escopo ou definição dentro de uma sprint. Por isso o cliente deve validar todos os requisitos no início do projeto, isso vai contribuir para evitar atrasos e até mesmo construções erradas.
- A XP enfatiza que não se deve fazer horas extras constantemente e, se isso ocorrer, existem problemas no projeto que devem ser resolvidos não com aumento de horas, mas com melhor planejamento. O Scrum enfatiza que equipes auto-organizáveis escolhem qual a melhor forma para completarem seu trabalho.
- O Scrum estabelece que os testes devem ocorrer o tempo todo durante o desenvolvimento, principalmente usando técnicas automatizadas. Na XP os testes podem ser realizados apenas na parte final de cada sprint, usando a técnica de refatoração, que busca validar todas as funcionalidades, pensando estrategicamente em como refatorar o código que está sendo implementado.



A XP é baseada numa série de práticas, a saber:

- Cliente Presente (*On-site customer*)
- Jogo do Planejamento (*The Planning Game*)
- Pequenos Lançamentos (*Small Releases*)
- Desenvolvimento Guiado pelos testes (*Test First Design*)

- Integração Contínua (*Continuous Integration*)
- Projeto Simples (*Simple Design*)
- Refatoração (*Refactoring*)
- Programação em pares (*Pair Programming*)
- Propriedade Coletiva (*Collective Ownership*)
- Padrões de Codificação (*Coding Standards*)
- Ritmo Sustentável (*40 Hour Week*)
- Metáfora (*Metaphor*)

De acordo com a prática de **Ritmo Sustentável**, a equipe não deve trabalhar mais do que 40 horas por semana, evitando-se horas extras.

Já no Scrum, a equipe deve determinar a forma de organizar seu horário de maneira autônoma.

Letra d.

012. (FCC/2014/TRT 13ª REGIÃO/PB/ANALISTA JUDICIÁRIO/TECNOLOGIA DA INFORMAÇÃO)

No *Scrum*, um projeto se inicia com uma visão simples do produto que será desenvolvido. A visão pode ser vaga a princípio e ir tornando-se clara aos poucos. O  então, transforma essa visão em uma lista de requisitos funcionais e não-funcionais para que, quando forem desenvolvidos, reflitam essa visão. Essa lista, chamada de , é priorizada pelo  de forma que os itens que gerem maior valor ao produto tenham maior prioridade.

Completa, correta e respectivamente, as lacunas I, II e III:

- Daily Scrum - Scrum Team - Sprint*
- Daily Scrum - Product Backlog - Sprint Planning Meeting*
- Product Owner - Sprint - Product Backlog*
- Scrum Team - Sprint Planning Meeting - Product Owner*
- Product Owner - Product Backlog - Product Owner*



O *Product Owner* é responsável pela identificação dos requisitos (funcionais e não-funcionais) que serão desenvolvidos. Estes requisitos compõem o *Product Backlog*, e serão desenvolvidos de acordo com a prioridade definida pelo *Product Owner* nas *Sprint Planning Meetings*.

Letra e.

013. (CESPE/2013/MPOG/TECNOLOGIA DA INFORMAÇÃO) Com relação às metodologias ágeis de desenvolvimento, julgue os itens a seguir. Do ponto de vista metodológico, o software livre é considerado uma abordagem similar aos métodos ágeis.



A banca considerou a assertiva como verdadeira no gabarito definitivo, provavelmente por considerar a “propriedade coletiva do código” em ambos os casos.

Certo.

014. (CESPE/SERPRO/ANALISTA/DESENVOLVIMENTO DE SISTEMAS/ENGENHARIA DE SOFTWARE/2013) Julgue os itens a seguir, acerca de metodologias ágeis de desenvolvimento. Kanban é um método de desenvolvimento de software que tem como uma de suas práticas o gerenciamento do fluxo de trabalho, que deve ser monitorado, medido e reportado a cada estado do fluxo.



Kanban é um **método de desenvolvimento de software** com fortes bases em práticas enxutas, e tem como objetivo otimizar o processo de desenvolvimento de software pré-existente.

Henrik destaca as seguintes práticas do Kanban:

- 1 - Visualizar o fluxo de trabalho (workflow);
- 2 - Limitar o trabalho em progresso;
- 3 - **Gerenciar e medir o fluxo.**

Segundo o autor, embora poucas, quando bem utilizadas essas práticas podem levar um time de desenvolvimento de software a ótimos resultados. David Anderson recomenda que se acrescente mais duas condições a essa lista de Kniberg, para se ter um processo ainda mais otimizado:

- 1 - Tornar políticas do processo explícitas;
- 2 - Usar modelos para avaliar Oportunidades de Melhoria.

Entende-se por **fluxo** a progressão visual dos itens de trabalho através do sistema Kanban.

Assim, uma de suas práticas é o **Gerenciamento do Fluxo**. Nesse contexto:

- visualize o fluxo de trabalho: divida o trabalho em partes, e use colunas nomeadas para ilustrar onde cada item se encontra no fluxo de trabalho;

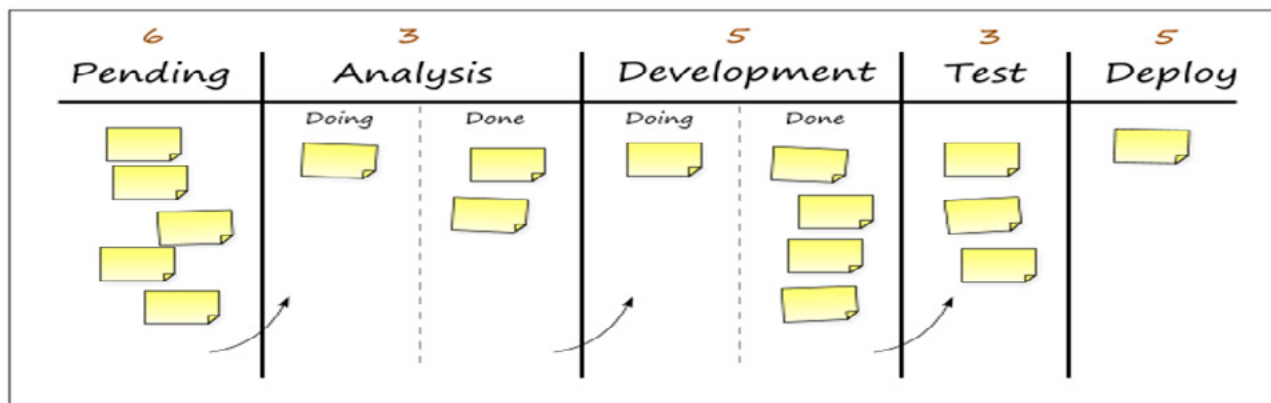


Figura. Quadro Kanban (Fonte: <http://www.kanbanblog.com/explained/>)

- limite o trabalho em progresso (*work in progress*), associando limites explícitos para quantos itens podem estar em progresso em cada estado do fluxo de trabalho;
- acompanhe o tempo de execução da tarefa e otimize o processo para tornar o tempo de execução o menor e mais previsível possível.

Segundo destaca Gomes (2013)

sempre que um item de trabalho parecer estar parado por muito tempo, sem fluir no processo, a equipe deve conversar a respeito para identificar a causa do problema, e então pensar em como melhorar a fluidez do processo.

Entre as causas possíveis para problemas que podem levar um item de trabalho a não fluir no Kanban podemos citar: dificuldades técnicas, cliente ou analistas de negócio indisponíveis para esclarecer dúvidas de negócio da equipe, requisitos pouco claros, ambiente de homologação indisponível, longas esperas por informações, etc.

Fonte:

Gomes, André Faria. Desenvolvimento Ágil com kanban, disponível em: http://www.devmedia.com.br/websys.5/webreader.asp?cat=6&artigo=2955&revista=javamagazine_84#a-2955.

Fonte: What is kanban? Em: <http://www.kanbanblog.com/explained>

Certo.

015. (FCC/ASSEMBLEIA LEGISLATIVA DO ESTADO DE SÃO PAULO/AGENTE TÉCNICO LEGISLATIVO/TECNOLOGIA DA INFORMAÇÃO/2010) São consideradas metodologias ágeis de desenvolvimento de software:

- XP e UP.
- SCRUM e DSDM.
- SCRUM e RUP.

- d) DSDM e Cascata.
- e) Cascata e PRINCE2.



O jeito aqui é eliminarmos o que temos certeza de que não é metodologia ágil, então vamos lá! O modelo em Cascata (*Waterfall*), também chamado de Clássico, é o mais tradicional processo de desenvolvimento de software, e então podemos descartar as letras (d) e (e). O **Rational Unified Process (RUP)** é um exemplo de modelo de processo de desenvolvimento baseado no *Unified Process* (Processo Unificado) desenvolvido pela Rational. Então já eliminados RUP e UP (Processo Unificado) que não tem relação com a metodologia ágil, o que excluirá as assertivas (a) e (d).

Por eliminação, chegamos à letra D.

O Scrum é um processo de desenvolvimento ágil de software baseado em grupos de práticas e papéis pré-definidos. Ele é um processo **iterativo e incremental para gerenciamento de projetos e desenvolvimento de sistemas**, em que cada *sprint* é uma iteração que segue um ciclo PDCA (*Plan, Do, Check, Act*) e entrega um incremento de software pronto.

Por fim, o **DSDM (Dynamic Systems Development Method - Método de Desenvolvimento de Sistemas Dinâmicos)**, segundo Pressman (2011, p. 96) é uma abordagem de desenvolvimento de software ágil que “oferece uma metodologia para construir e manter sistemas que atendem restrições de prazo apertado através do uso da prototipagem incremental em um ambiente de projeto controlado.

Letra b.

016. (CESPE/BANCO DA AMAZÔNIA/TÉCNICO CIENTÍFICO/ÁREA: TECNOLOGIA DA INFORMAÇÃO/ARQUITETURA DE TECNOLOGIA/2010) O Scrum é utilizado, como função primária, para o gerenciamento de projetos de desenvolvimento de software, mas também tem sido usado como *extreme programming* e outras metodologias de desenvolvimento. Teoricamente, o Scrum pode ser aplicado em qualquer contexto no qual um grupo de pessoas necessite trabalhar juntas para atingir um objetivo comum.



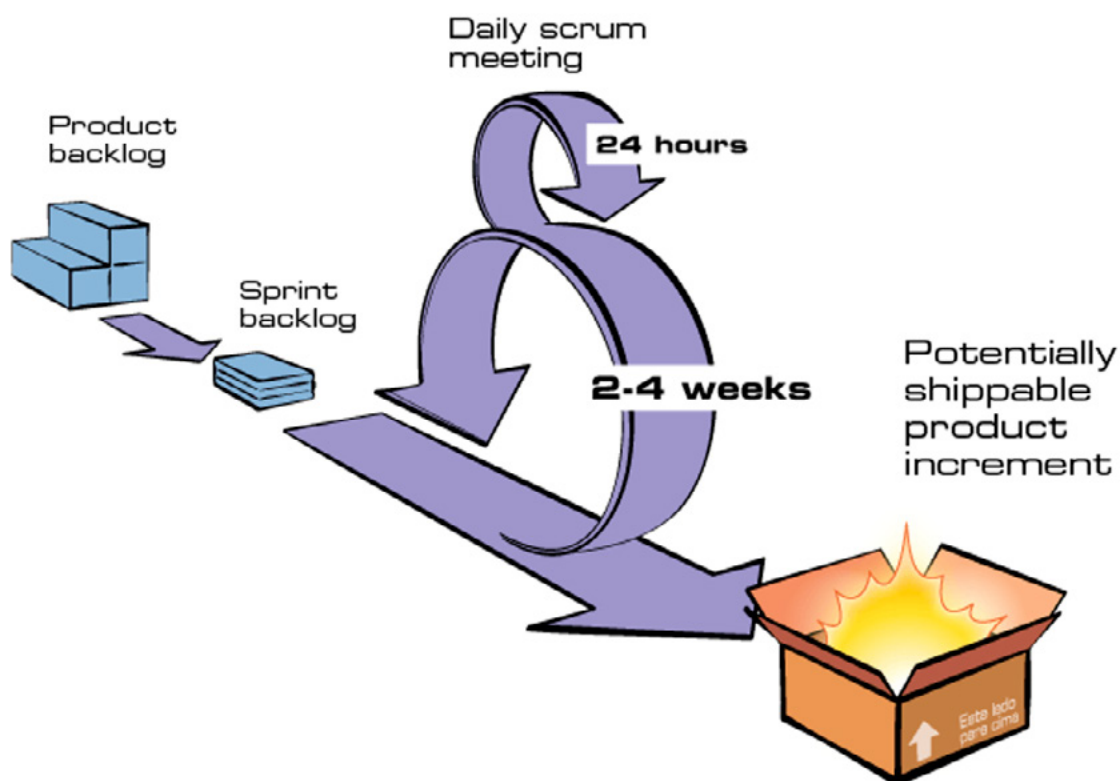
Define-se **Scrum** como uma **metodologia para desenvolvimento ágil e Gerenciamento de Projetos**. Esta metodologia foi concebida como uma forma de gerência de projetos em empresas automobilísticas e de produtos de consumo, a partir da observação de que projetos usando equipes pequenas e multidisciplinares produziram os melhores resultados, e associaram estas equipes altamente eficazes à formação Scrum do Rugby.

De acordo com o artigo apresentado no site <http://improveit.com.br/scrum>, na metodologia Scrum, os projetos são divididos em ciclos (tipicamente mensais) chamados de Sprints.

O Sprint representa um intervalo de tempo no qual uma série de atividades deve ser executada. Como toda metodologia ágil de desenvolvimento de software, ele é iterativo, e cada iteração, é chamada de Sprint.

A utilização do Scrum ocorre da seguinte forma:

- As funcionalidades de um projeto são mantidas em uma lista chamada *Product Backlog*;
- Ao iniciar-se um Sprint, faz-se uma reunião de planejamento (Sprint Planning Meeting) na qual o *Product Owner* prioriza as funcionalidades do *Product Backlog* e a equipe seleciona as atividades que ela será capaz de implementar;
- As tarefas do respectivo Sprint são transferidas do *Product Backlog* para o *Sprint Backlog*.
- Diariamente, a equipe faz uma breve reunião (Daily Scrum), em geral pela manhã, visando disseminar conhecimento sobre o trabalho realizado no dia anterior, identificar impedimentos e priorizar o trabalho do dia; e
- No término de um *Sprint*, a equipe apresenta as funcionalidades implementadas em uma *Sprint Review Meeting*. Após isso, faz-se uma *Sprint Retrospective* e a equipe parte para o planejamento do próximo *Sprint*.
- Reinicia-se assim o ciclo.



Certo.

017. (FUMARC/CEMIG/ANALISTA DE TI JUNIOR/2010) Sobre modelos de processo de desenvolvimento de software, assinale a alternativa INCORRETA:

- a) O Scrum é um processo de desenvolvimento ágil de software baseado em grupos de práticas e papéis pré-definidos. Ele é um processo iterativo e incremental para gerenciamento de projetos e desenvolvimento de sistemas, onde cada *sprint* é uma iteração que segue um ciclo PDCA (*Plan, Do, Check, Act*) e entrega um incremento de software pronto.
- b) O design centrado no usuário (UCD) é uma abordagem do processo de desenvolvimento de software baseada no entendimento explícito dos usuários, tarefas, e tem como objetivo principal o casamento entre o modelo conceitual embutido no sistema pelo projetista e o modelo mental do usuário.
- c) Programação extrema (XP – *extreme programming*) é um processo de desenvolvimento ágil baseado em *feedback* rápido, e simplicidade; com enfoque explícito em tempo, custo e qualidade no desenvolvimento, que são alcançados através de uma definição rígida do escopo das funcionalidades da aplicação.
- d) O modelo em espiral é um processo de desenvolvimento de software que intercala etapas de projeto e prototipação, combinando conceitos de desenvolvimento *top-down* e *bottom-up*, e permitindo, desta forma, análise de riscos e estimativas do progresso do trabalho mais realistas.



a) Certa. **Scrum** é um framework de processo ágil utilizado para gerenciar e controlar o desenvolvimento de um produto de software através de práticas iterativas e incrementais. É composto por um conjunto de boas práticas de gestão que admite ajustes rápidos, acompanhamento e visibilidade constantes e planos realísticos.

- **Scrum é um processo bastante leve para gerenciar e controlar projetos de desenvolvimento de software e para a criação de produtos.**

b) Certa. O **design centrado no usuário (UCD)** é uma forma de se trabalhar o desenvolvimento de qualquer produto a partir das necessidades dos usuários. Ele (UCD) envolve a aplicação de metodologias de usabilidade essenciais ao desenvolvimento de sistemas interativos e produtos. Alguns conceitos:

Donald A. Norman → *The Design of Everyday Things*

Princípios de design que – quando seguidos – dão respostas aos usuários tornando o uso dos dispositivos mais fácil

Albert N. Badre → *Shaping Web Usability*

Facilidade de uso, e facilidade de aprendizado

Jeffrey Rubin → *Handbook of Usability Testing*

Um conjunto de quatro fatores reunidos em um dispositivo: 1) capacidade de ser usado com sucesso; 2) facilidade de ser usado; 3) capacidade de o usuário aprender a usar o dispositivo de forma simples e rápida; 4) Provocar satisfação visual ao usuário.

Brian Shackel → *Usability—context, framework, definition, design and evaluation*

Capacidade, em termos funcionais humanos, de um sistema ser usado com facilidade e de forma eficiente.

Jesse James Garrett → *The Elements of User Experience*

Pensar em usabilidade é pensar em produtos fáceis de usar

Kreta Chandler e Karen Hyatt → *Customer-centered Design*

Efetividade, eficiência e satisfação com a qual usuários conseguem atingir seus objetivos ao utilizar um dispositivo

Mark Pearrow → *Web Site Usability Handbook*

A ciência de aplicação de metodologias ao design para a criação de dispositivos fáceis de usar, de fácil aprendizado e que sejam úteis com o menor índice de desconforto possível

Jakob Nielsen → *Usability Engineering*

Um conjunto de propriedades de uma interface que reúne os seguintes componentes: 1) Fácil aprendizado; 2) Eficiência; 3) Capacidade de memorização; 4) Baixo índice de erros; 5) Satisfação e prazer ao uso.

c) Errada. **Programação extrema (XP – extreme programming)** não prevê a definição rígida do escopo das funcionalidades da aplicação, uma vez que é uma metodologia voltada para projetos cujos requisitos são vagos e mudam com frequência.

d) Certa. A principal inovação do **Modelo Espiral** é guiar o processo de desenvolvimento gerado a partir de um *metamodelo* com base em análise de riscos e planejamento que é realizado durante toda a evolução do desenvolvimento.

Letra c.

018. (FCC/TRF 4ª REGIÃO/ANALISTA JUDICIÁRIO/TECNOLOGIA DA INFORMAÇÃO/2010) Na fase de desenvolvimento do Scrum, o software é desenvolvido em processos iterativos denominados

- a) *Building Products*.
- b) *Product Backlog*.
- c) Sprint.
- d) *Product Owner*.
- e) *Product Backlog Cycle*.



O **Scrum** é um **processo de desenvolvimento ágil de software** baseado em grupos de práticas e papéis predefinidos.

Ele é um processo **iterativo** e **incremental** para gerenciamento de projetos e desenvolvimento de sistemas, em que cada **sprint** é uma iteração que segue um ciclo PDCA (*Plan, Do, Check, Act*) e entrega um incremento de software pronto.

Letra c.

019. (CEBRASPE/CESPE/ANALISTA DE PREVIDÊNCIA COMPLEMENTAR/FUNPRESP-EXE/TECNOLOGIA/2022) Acerca da engenharia de software, julgue o item que se segue.

Na fase de desenvolvimento do Scrum, os requisitos são escritos no *product backlog*.



Artefatos Oficiais	Descrição
Product Backlog	Uma lista ordenada (por valor, risco, prioridade, etc.) de requisitos ou funcionalidades que o produto deve conter criada pela Equipe Scrum.

Conforme visto, **os requisitos são escritos no Product Backlog**. O erro da questão está evidente ao listar que os requisitos são escritos na fase de desenvolvimento, o que não é verdade. Na fase de desenvolvimento da Scrum já será utilizado o levantamento de requisitos!

Errado.

020. (UFMT/PERITO OFICIAL/POLITEC MT/CRIMINAL/CIÊNCIA DA COMPUTAÇÃO OU INFORMÁTICA/2022) No SCRUM, o papel responsável por garantir o máximo de valor agregado ao produto é:

- a) do Cliente.
- b) do *Scrum Master*.
- c) da Equipe.
- d) do *Product Owner*.
- e) do Desenvolvedor.



Os principais papéis do Scrum são: **Product Owner (PO)**, **Scrum Master (SM)** e **Development Team (DT)**.

Papéis do Scrum	Descrição
Product Owner (PO)	Dono do Produto. É uma pessoa e não um comitê. Ele pode representar o desejo de um comitê no <i>Product Backlog</i>, mas aqueles que quiserem uma alteração nas prioridades dos itens de <i>backlog</i> devem convencer o Product Owner. É o responsável por maximizar o valor do produto e do trabalho do Time de Desenvolvimento. Como isso é feito pode variar amplamente através das organizações, Times Scrum e indivíduos.

Letra d.

REFERÊNCIAS

ATLASSIAN. **O que é Kanban**. Disponível em: <www.atlassian.com/agile/kanban

>. Acesso em: jun. 2021.

GOMES, A. F. **Desenvolvimento Ágil com kanban**. Disponível em: <http://www.devmedia.com.br/websys.5/webreader.asp?cat=6&artigo=2955&revista=javamagazine_84#a-2955

.> Acesso em: jun. 2021.

KRUCHTEN, P., **Introdução ao RUP Rational Unified Process**, 2. ed., Rio de Janeiro: Ciência Moderna, 2003.

Manifesto Ágil. Disponível em: <<https://agilemanifesto.org/iso/ptbr/manifesto.html>

>. Acesso em: jun. 2021.

PFLEEGER, S. L. **Engenharia de Software: Teoria e Prática**. 2.ed., São Paulo: Prentice Hall, 2004.

POPPENDIECK, M.; CUSUMANO, M. A. **Lean Software Development: A Tutorial**. **IEEE Software**, v. 29, n. 5, p. 26-32, 2012.

POSSAMAI, Edson. **A contribuição do Lean Manufacturing na Obtenção de Vantagem Competitiva: Estudo de Caso em Duas Empresas do Setor Moveleiro na Região de São Bento do Sul/SC**. 2011. 116 f. Dissertação (Mestrado) – Curso de Mestrado em Engenharia de Produção, Sociedade Educacional de Santa Catarina Instituto Superior Tupy, Joinville/SC, 2011.

PRESSMAN, R. S. **Engenharia de Software: Uma Abordagem Profissional**, 7. ed. Porto Alegre: Editora Mc GrawHill, 2011.

Princípios por Trás do Manifesto Ágil. Disponível em: <<https://agilemanifesto.org/iso/ptbr/principles.html>>. Acesso em: jun. 2021.

STONE, K. B. **Four Decades of Lean: a Systematic Literature Review**. **International Journal of LeanSix Sigma**, v. 3, n. 2, p. 112-132, 2012.

QUINTÃO, P. L. **Tecnologia da Informação para Concursos**. 2022.

SCHWABER, K. e Beedle, M. **Agile Software Development with Scrum**, Prentice Hall, 2002.

SCHWABER, K; Beedle, M. **Agile Software Development**, Cockburn Highsmith Series Editors, Alistair Cockburn, 2000.

SOMMERVILLE, I., **Engenharia de Software**, 9. ed., São Paulo: Pearson Addison – Wesley, 2011.

SOTILE, M. **O Que é Scrumban?** Disponível em: <<https://blog.pmtech.com.br/scrumban/#>>. 2018. Acesso em: jun 2021.

SBROCCO, J. H. T. de C.; MACEDO, P. C. **Metodologias ágeis: engenharia de software sob medida**. 1. ed. – São Paulo: Érica, 2012.

Abra



caminhos



crie

futuros

gran.com.br

